

UTFPR - UNIVERSIDADE TECNOLÓGICA FEDERAL DO
PARANÁ

Bacharelado em Engenharia de Software - 6º Período

DISCIPLINA: Oficina de Integração 1 - ES46F-ES61

Professor: Eduardo Cotrin Teixeira

Documento de Projeto de Software

Cursinho Prisma

Gustavo Pukanski Schatzmann
Igor Busquim de Moraes
Luiz Fernando Moreira Domenico
Orlando Cardoso Neto
Rafael Spitzer Cardoso da Silva

Cornélio Procópio
2026

Sumário

1	Introdução	3
1.1	Contexto	3
1.2	Justificativa	3
1.3	Proposta	4
1.4	Organização do Documento	4
2	Descrição Geral do Sistema	5
2.1	Objetivos (Gerais e Específicos)	5
2.1.1	Objetivo Geral	5
2.1.2	Objetivos Específicos	5
2.2	Limites e Restrições	5
2.3	Limites	5
2.4	Restrições	6
2.5	Descrição dos Usuários do Sistema	6
3	Desenvolvimento do Projeto	7
3.1	Tecnologias e ferramentas	7
3.1.1	Serviço de Inteligência Artificial	7
3.1.2	Aplicação Web e API	7
3.1.3	Banco de Dados e Armazenamento	7
3.1.4	Gerenciamento, Qualidade e Documentação	7
3.2	Metodologia de desenvolvimento	7
3.3	Cronograma previsto	8
4	Requisitos do Sistema	9
4.1	Requisitos Funcionais	9
4.2	Requisitos Não-funcionais	11
4.3	Diagramas de Casos de Uso	11
4.4	Protótipos de Telas	13
4.4.1	Tela de Login	13
4.4.2	Dashboard do Professor	13
4.4.3	Tela de Cadastro de Questões	14
4.4.4	Tela de Criar Listas	15
4.4.5	Tela do Banco de Questões	15
4.4.6	Tela de Gerenciamento de Frequência	16
4.4.7	Dashboard do Aluno	17
4.4.8	Tela de Resolução do Exercício	17

4.4.9	Painel Administrativo	18
5	Análise do Sistema	19
5.1	Modelo do Banco de Dados	19
5.1.1	Dicionário de Dados	20
5.2	Diagrama de Classes	21
5.3	Diagramas de Atividades	22
6	Implementação	32
6.1	Descrição do código	32
6.1.1	Back-End (.NET 9 / C#)	32
6.1.2	Front-End (Angular 21+)	33
6.1.3	Inteligência Artificial (Python / FastAPI / Ollama llama3.2:3b)	33
6.2	Implantação	34
6.2.1	Requisitos de Software	34
6.2.2	Passo 1 — Instalar e configurar o Ollama	34
6.2.3	Passo 2 — Clonar o repositório	34
6.2.4	Passo 3 — Configurar as variáveis de ambiente	34
6.2.5	Passo 4 — Subir o back-end com Docker	35
6.2.6	Passo 5 — Subir o front-end	36
6.2.7	Passo 6 — Acessar o sistema	36
6.2.8	Solução de Problemas Comuns	36
6.3	Telas Principais	36
6.3.1	Login	36
6.3.2	Dashboard do Professor	37
6.3.3	Cadastrar Questão	37
6.3.4	Banco de Questões	38
6.3.5	Criar Lista	39
6.3.6	Gerenciar Turmas	39
6.3.7	Dashboard do Aluno	40
6.3.8	Resolução de Exercícios	41
6.3.9	Banco Livre	41
6.3.10	Painel Administrativo	42
7	Considerações Finais	43
7.1	Sucessos	43
7.2	Limitações	43
7.3	Trabalhos Futuros	43
8	Bibliografia	44

1 Introdução

1.1 Contexto

O projeto será implementado no Cursinho Prisma, um projeto de extensão da Universidade Tecnológica Federal do Paraná (UTFPR – Câmpus Cornélio Procópio) focado na preparação gratuita de estudantes para o vestibular e o ENEM. A instituição opera em um cenário de alta demanda social e capacidade estrutural limitada, ofertando apenas uma turma por semestre de 40 a 45 alunos, com cerca de 80 a 90 alunos aplicando para o projeto.

Devido a essa limitação, o processo de admissão é rigoroso. Historicamente o critério era o de maior média escolar; a triagem evoluiu, a partir da edição de 2026/02, para a aplicação de uma prova classificatória de múltipla escolha. A alocação das vagas obedece a um sistema de prioridades: a preferência é destinada aos alunos matriculados no 3º ano do Ensino Médio, seguidos por alunos de outras séries do ensino médio e, por fim, estudantes egressos.

Uma vez inserido no projeto, o aluno ingressa em um ecossistema com regras administrativas e pedagógicas rígidas. Administrativamente, exige-se uma frequência mínima obrigatória de 75%, e o descumprimento desta meta resulta na expulsão sumária do estudante. Pedagogicamente, a instituição não dispõe de infraestrutura e/ou recursos humanos para oferecer atividades de reforço ou turmas extras de nivelamento. O aluno depende exclusivamente da carga horária regular e de sua própria capacidade de autodidatismo para superar defasagens nas disciplinas. Atualmente, os processos de controle (frequência) e apoio (materiais) operam de forma puramente manual e descentralizada.

1.2 Justificativa

O desenvolvimento deste software justifica-se pela necessidade de mitigar dois riscos críticos da operação atual do Cursinho Prisma: a “cegueira de dados” pedagógica e a vulnerabilidade do controle administrativo manual.

No aspecto pedagógico, o problema central é a ineficiência no processo de curadoria de exercícios e a total ausência de telemetria sobre o aprendizado. As questões são extraídas manualmente pelos professores a partir de provas anteriores de diversos vestibulares e do ENEM. Esse processo de triagem, categorização e formatação gera uma sobrecarga que inviabiliza a criação constante de listas personalizadas. Sem aulas de reforço para diagnosticar defasagens, os alunos e a coordenação operam no escuro. Um estudante pode ter um déficit em uma matéria específica (ex: Logaritmos), mas essa lacuna passará despercebida até o dia da prova final, resultando em nivelamento acadêmico deficiente e provável evasão.

No aspecto administrativo, o controle da regra de 75% de frequência feito manual-

mente aumenta o risco de falhas operacionais, registros incorretos e expulsões indevidas (ou a não detecção de evasão iminente).

Ao automatizar o fluxo de avaliações e instituir uma cultura de dados, o sistema converte a simples ação de “responder questões” em um motor de telemetria educacional. Isso elimina o atrito operacional do professor na criação de listas direcionadas e fornece ao aluno métricas de suas dificuldades, compensando tecnologicamente a ausência de tutoria individualizada.

1.3 Proposta

A aplicação é uma plataforma web pensada para reduzir o trabalho operacional na criação de avaliações e melhorar o acompanhamento do desempenho acadêmico. A ideia principal é conectar a rotina dos professores com a prática dos alunos, usando automação para diminuir tarefas manuais, principalmente na organização e categorização de questões.

No módulo do docente, o foco está na gestão e no uso de dados para facilitar o dia a dia. O sistema oferece uma interface para cadastro de questões com suporte a texto e imagens. Um dos recursos é a integração com inteligência artificial, que analisa o conteúdo das questões e sugere automaticamente a classificação por disciplina e nível de dificuldade. Também há ferramentas para gerenciar turmas e montar listas de exercícios de forma rápida, com base em filtros. Os painéis analíticos permitem acompanhar o desempenho das turmas e dos alunos, ajudando a identificar onde estão as principais dificuldades.

O módulo do aluno é voltado para a prática e o acompanhamento do próprio desempenho. O aluno pode acessar tanto o banco geral de questões quanto listas específicas enviadas pelos professores. O sistema conta com um painel individual que mostra a frequência e o histórico de acertos e erros, permitindo que o aluno identifique seus pontos fortes e onde precisa melhorar.

1.4 Organização do Documento

O capítulo 1 apresenta o contexto do Cursinho Prisma, a justificativa do problema a ser resolvido e a proposta de solução baseada em IA e gestão de dados. O capítulo 2 detalha os objetivos gerais e específicos do sistema, expõe as limitações técnicas e define os papéis dos usuários. O capítulo 3 detalha as tecnologias, a metodologia de desenvolvimento e o cronograma. O capítulo 4 especifica os requisitos funcionais e não-funcionais. O capítulo 5 apresenta a modelagem do banco de dados e os diagramas de classe e de atividade. O capítulo 6 detalha a implementação do projeto, com o diagrama de componentes, as instruções de implantação e as telas principais do sistema finalizado. O capítulo 7 apresenta as considerações finais. O capítulo 8 é dedicado a bibliografia usada no projeto

2 Descrição Geral do Sistema

2.1 Objetivos (Gerais e Específicos)

2.1.1 Objetivo Geral

Desenvolver e implantar um sistema web para gestão educacional focado em avaliação de desempenho e criação de listas de exercícios, sustentado por um banco de questões alimentado colaborativamente através de uma interface assistida por IA para automação da categorização e da dificuldade.

2.1.2 Objetivos Específicos

Metas e ações que, somadas, permitem alcançar o objetivo geral:

- Desenvolver uma interface web otimizada para inserção rápida de questões;
- Integrar um modelo de IA local (como o Ollama) à API para processar textos de questões inseridas e retornar sugestões estruturadas de categorização e dificuldade;
- Desenvolver o módulo do professor, permitindo o agrupamento de questões em listas e a gestão de turmas;
- Desenvolver o módulo do aluno para resolução de exercícios;
- Implementar dashboards de proficiência baseados no histórico de resoluções.

2.2 Limites e Restrições

2.3 Limites

- **Limitação de extração:** O sistema não realizará extração autônoma, leitura de PDFs ou OCR de provas. Toda questão deve ser inserida manualmente pelo usuário;
- **Limitação da IA:** A IA atuará estritamente como um assistente de preenchimento. O usuário terá a palavra final para aceitar, alterar ou recusar a categoria e a dificuldade proposta pelo modelo antes de salvar no banco;
- **Interação síncrona:** Está fora do escopo a criação de fóruns, chats em tempo real, ou transmissão de aulas;
- **Integração externa:** O sistema não fará integração com sistemas acadêmicos governamentais ou plataformas externas de vestibulares.

2.4 Restrições

- A extração automática de questões a partir de PDFs não será implementada nesta versão, podendo ser uma funcionalidade futura;
- A acurácia de categorização não será de 100%, sendo necessária a intervenção manual humana antes da persistência no banco de dados;
- Por falta de recursos, o modelo de Ollama é executado localmente no ambiente do usuário
- Pela falta de recursos, o sistema irá se restringir ao uso de ferramentas gratuitas ou com tiers gratuitos.

2.5 Descrição dos Usuários do Sistema

- **Aluno:** Acessa a plataforma web para resolver as listas designadas pelos professores, treina no banco livre e acompanha o seu desempenho pessoal nas disciplinas, em disciplinas e matérias;
- **Professor/Monitor:** Acessa a plataforma web para analisar o desempenho das turmas por meio do dashboard, identifica discrepâncias no desempenho, cadastra novas questões no banco, gera listas de exercícios personalizadas e direcionadas consultando o banco de questões;
- **Administrador (Coordenação):** Gerencia os cadastros de professores e alunos e tem acesso ao log do sistema.

3 Desenvolvimento do Projeto

3.1 Tecnologias e ferramentas

3.1.1 Serviço de Inteligência Artificial

- **Linguagem:** Python com FastAPI, atuando como uma API interna;
- **Categorização semântica e de dificuldade:** Ollama;
- **Execução:** Local via Docker.

3.1.2 Aplicação Web e API

- **Linguagem backend:** ASP .NET 9;
- **Linguagem frontend:** Angular 21+;
- **Containerização:** Docker.

3.1.3 Banco de Dados e Armazenamento

- **SGBD relacional:** PostgreSQL;
- **Autenticação e Armazenamento:** Supabase.

3.1.4 Gerenciamento, Qualidade e Documentação

- **Versionamento:** Git com repositório no GitHub;
- **Gerenciamento, gestão e acompanhamento do cronograma:** Jira;
- **Testes de API:** Bruno;
- **Comunicação:** Whatsapp;

3.2 Metodologia de desenvolvimento

O projeto adota a metodologia de Desenvolvimento Iterativo e Incremental, gerenciado através de um fluxo contínuo baseado em Kanban. A exigência de rituais síncronos fixos mostrou-se incompatível com a realidade da equipe, composta por integrantes que trabalham em período integral e frequentam as aulas da graduação no período noturno.

A equipe opera em regime de desenvolvimento assíncrono. O escopo foi desmembrado no Jira, e o fluxo de integração funciona sob demanda. A comunicação diária é realizada via WhatsApp. O código é submetido ao repositório no GitHub conforme os desenvolvedores concluem suas tarefas, garantindo que o sistema evolua e atenda aos prazos estabelecidos pelo cronograma da disciplina.

3.3 Cronograma previsto

Tabela 1: Cronograma de desenvolvimento

Período	Atividades
23/03 a 05/04	Fundação e Planejamento: levantamento de requisitos, protótipos de telas, modelagem do banco de dados, setup inicial de repositórios
06/04 a 19/04	Arquitetura e Base Técnica (Entrega 1 – 07/04): configuração de infraestrutura, estrutura inicial do front-end, setup da IA local
20/04 a 03/05	Base do Sistema: CRUD inicial, primeiros microsserviços, integração inicial de front e back
04/05 a 10/05	Módulo Professor: microsserviços de Professor e Turmas, CRUD completo
11/05 a 17/05	Integração com IA (Entrega 2 – 12/05): microsserviço de Questões, integração com FastAPI da IA, autenticação JWT
18/05 a 24/05	Listas e Resolução: criação de listas com filtros, tela de resolução de exercícios, tela de frequência
25/05 a 07/06	Dashboards e Métricas: dashboard do aluno e professor, métricas de desempenho
08/06 a 21/06	Otimização e Testes: refinamento da IA, otimização de queries, ajustes de UX/UI
22/06 a 28/06	Produção Final (Entrega 3 – 23/06): deploy completo, documentação final

4 Requisitos do Sistema

4.1 Requisitos Funcionais

Tabela 2: Requisitos funcionais do sistema

ID	Funcionalidade	Prioridade
RF01	O sistema deve prover uma interface para inserção manual do enunciado, alternativas e gabarito, além de upload de imagem de apoio	Essencial
RF02	Ao preencher o enunciado, o sistema deve acionar a IA para sugerir a disciplina, matéria e dificuldade	Essencial
RF03	O professor deve poder aceitar, editar ou recusar as sugestões da IA antes de salvar a questão	Essencial
RF04	Professores e administradores devem poder listar, editar e desativar questões já salvas no banco	Essencial
RF05	O sistema deve gerenciar autenticação e autorização para três perfis: Administrador, Professor e Aluno	Essencial
RF06	Professores devem poder criar turmas e adicionar alunos a elas, permitindo que um mesmo aluno esteja em múltiplas turmas simultaneamente	Importante
RF07	Professores devem poder buscar questões no banco através de filtros e agrupá-las em listas atribuídas a turmas específicas	Importante
RF08	Alunos devem poder abrir listas ou o banco livre, selecionar a alternativa e receber a correção automática	Essencial
RF09	O sistema deve exibir gráficos para o aluno detalhando sua porcentagem de acertos/erros agrupados por macro e micro disciplinas	Essencial
RF10	O sistema deve exibir relatórios de proficiência mostrando as maiores lacunas de aprendizado da turma e alunos individuais	Essencial
RF11	A coordenação deve poder gerenciar contas de usuários e visualizar logs do sistema	Essencial
RF12	O sistema deve fornecer uma interface para o professor registrar a presença ou falta dos alunos	Desejável
RF13	No dashboard do aluno deve haver um indicador exibindo seu percentual de presença na turma principal	Desejável

- **RF01 - Interface de Cadastro Manual e Upload:** O sistema deve fornecer um formulário interativo no front-end para que professores ou monitores insiram o texto bruto da questão, as opções de múltipla escolha e a indicação do gabarito correto. Além disso, deve haver um componente de upload que permita anexar uma imagem à questão, enviando o arquivo diretamente ao Supabase e salvando a URL no banco de dados.
- **RF02 - Classificação Assistida por IA:** No momento em que o usuário preenche o enunciado da questão no formulário, o front-end deve disparar uma requisição

assíncrona. A API ASP .NET repassará o texto para a API Python, que utilizará o modelo Ollama para analisar o contexto. O sistema deve retornar um JSON com a sugestão da disciplina macro, matéria micro e nível de dificuldade, baseando-se estritamente nas categorias existentes no banco de dados.

- **RF03 - Revisão Humana:** A interface não deve salvar a questão automaticamente após o retorno da IA. As sugestões geradas pelo RF02 devem preencher os campos de seleção (dropdown) da tela, servindo apenas como um facilitador. O usuário terá a obrigatoriedade de revisar, alterar ou confirmar essas categorias manualmente antes de submeter o formulário para persistência no banco de dados.
- **RF04 - Gestão de Questões (CRUD Base):** O sistema deve permitir operações completas de manipulação do acervo. Usuários com perfil de Professor ou Administrador poderão visualizar todas as questões em formato de lista (com filtros), editar o conteúdo de questões e desativar ou excluir questões desatualizadas do banco.
- **RF05 - Autenticação e Autorização:** O sistema deve possuir uma tela de login centralizada. Utilizando tokens JWT, o back-end validará as credenciais e as permissões. A interface web deve se adaptar ao perfil logado: alunos não podem acessar telas de cadastro de questões, e professores não podem acessar o painel de criação de usuários da administração.
- **RF06 - Gestão de Turmas:** O perfil professor deve ter acesso a um painel para criação e gerenciamento de "Turmas"(grupos lógicos de alunos). O professor poderá buscar alunos cadastrados no sistema e vinculá-los a uma ou mais turmas, permitindo que um mesmo aluno participe simultaneamente de diferentes turmas (por exemplo, turma regular e turmas de reforço), facilitando a organização e distribuição de atividades.
- **RF07 - Criação de Listas de Exercícios:** O professor deve acessar uma interface de busca no banco de questões consolidado. Através de filtros (por disciplina, matéria e dificuldade), ele poderá selecionar múltiplas questões e agrupá-las em uma lista. Esta lista será então atribuída a uma ou mais turmas cadastradas (RF06).
- **RF08 - Resolução de Exercícios (Aluno):** O aluno logado deve acessar um painel com suas listas pendentes. Ao abrir uma lista ou buscar questões no banco livre, ele verá a questão formatada, selecionará uma alternativa e enviará a resposta. O sistema deve registrar a tentativa no banco de dados e retornar o gabarito, indicando acerto ou erro.
- **RF09 - Dashboard de Autodesempenho (Aluno):** O sistema deve processar o histórico de resoluções do aluno logado e renderizar gráficos visuais. O painel

deve destacar a taxa global de proficiência e detalhar os dados por macro e micro disciplinas, orientando o estudo individual.

- **RF10 - Dashboard de Desempenho e Defasagem (Professor):** O sistema deve consolidar as métricas de todos os alunos de uma turma e gerar relatórios visuais para o professor. O objetivo é identificar tendências de erro coletivas e permitir intervenções pedagógicas direcionadas, além de possibilitar a análise individual de alunos com risco de evasão.
- **RF11 - Gestão Administrativa de Perfis e Estrutura:** A coordenação terá um painel exclusivo para realizar o CRUD de usuários e gerenciar a árvore de disciplinas, inserindo novas matérias no banco que orientarão tanto os filtros quanto as restrições da IA.
- **RF12 - Gerenciamento de Frequência (Chamada):** O sistema deve fornecer uma interface rápida para o professor selecionar uma turma, uma data específica e registrar a presença ou falta dos alunos, persistindo o histórico no banco de dados.
- **RF13 - Consulta de Frequência (Aluno):** No painel do aluno, deve haver um indicador visual exibindo seu percentual de presença acumulado, calculado a partir dos registros inseridos pelos professores.

4.2 Requisitos Não-funcionais

Tabela 3: Requisitos não funcionais do sistema

ID	Requisito	Categoria
RNF01	O tempo de espera pela sugestão da IA não deve travar a tela, exibindo indicação visual de carregamento	Usabilidade
RNF02	O banco de dados deve impor restrições de chave estrangeira entre micro e macro disciplinas	Confiabilidade
RNF03	O sistema não deve expor a comunicação direta com a IA; o front-end deve fazer requisições apenas à API	Segurança
RNF04	Senhas devem ser protegidas no banco utilizando algoritmo de hashing	Segurança

4.3 Diagramas de Casos de Uso

O diagrama de casos de uso ilustra as interações dos atores com o sistema, conforme a Figura 1.

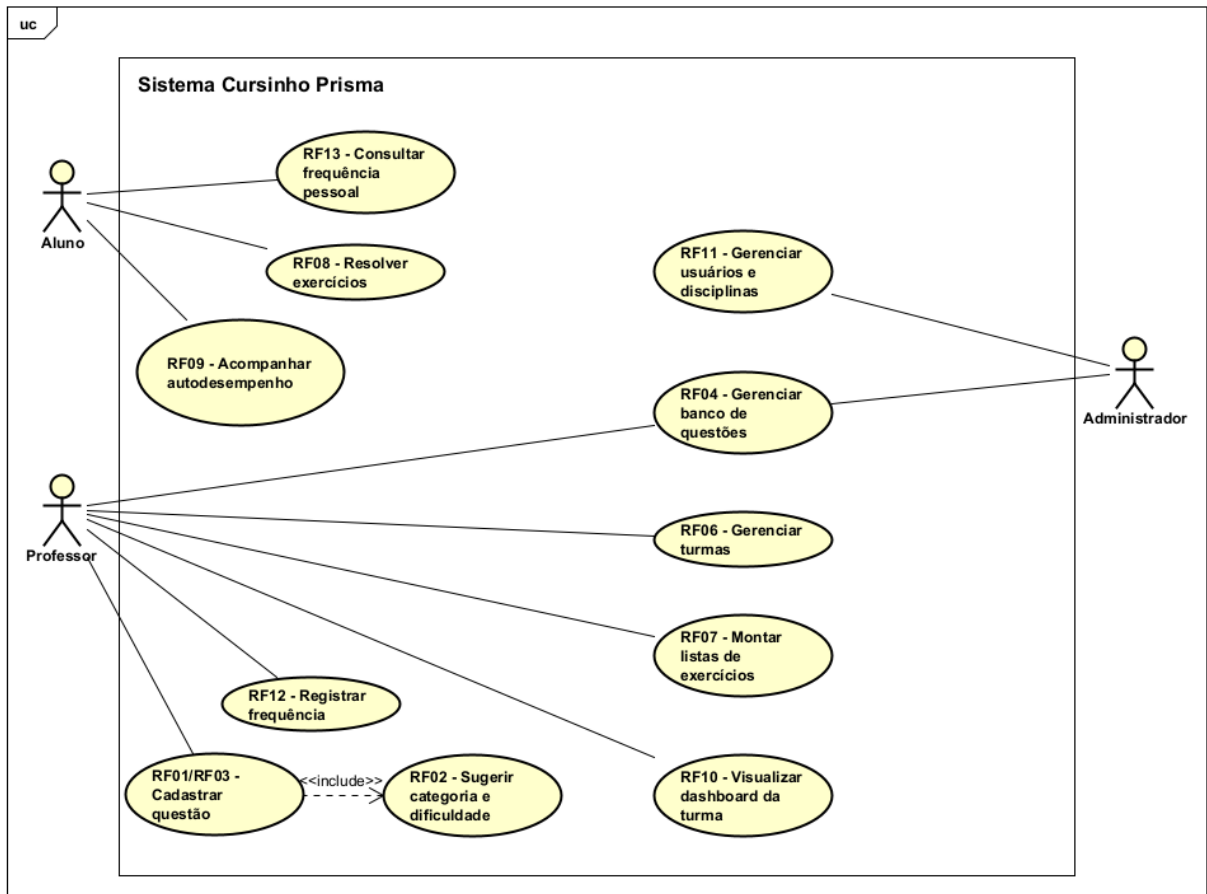


Figura 1: Diagrama de casos de uso representando as interações dos alunos, professores e administradores com o sistema Cursinho Prisma.

4.4 Protótipos de Telas

4.4.1 Tela de Login

Objetivo: Autenticar os usuários e direcioná-los para o ambiente correto de acordo com seu perfil.

Dinâmica de Navegação: É a tela inicial do sistema, apresentada na Figura 2. A partir dela, dependendo da aba selecionada e da validação das credenciais, o usuário é redirecionado para o seu respectivo dashboard. Também possui link para recuperação de senha.

Requisitos Funcionais Atendidos: RF05.

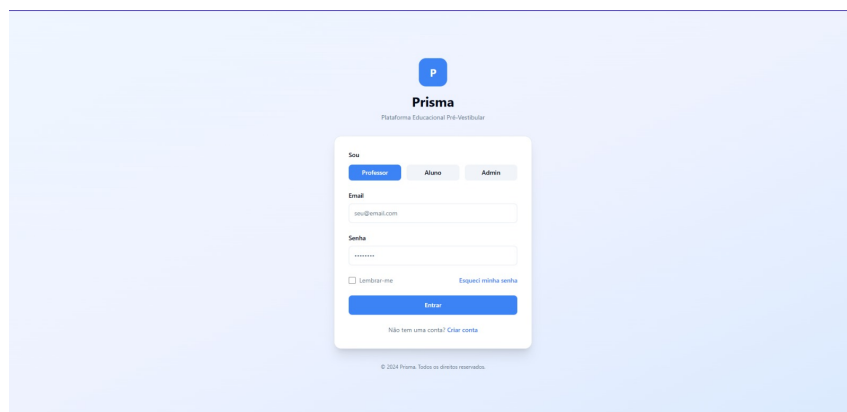


Figura 2: Tela de login.

4.4.2 Dashboard do Professor

Objetivo: Fornecer um painel gerencial resumido para o educador, com métricas-chave como total de alunos, questões cadastradas, desempenho médio das turmas e alunos em atenção.

Dinâmica de Navegação: É a tela principal acessada após o login do perfil professor, apresentada na Figura 3. Através do menu lateral, permite navegar para todas as funcionalidades do sistema.

Requisitos Funcionais Atendidos: RF10.

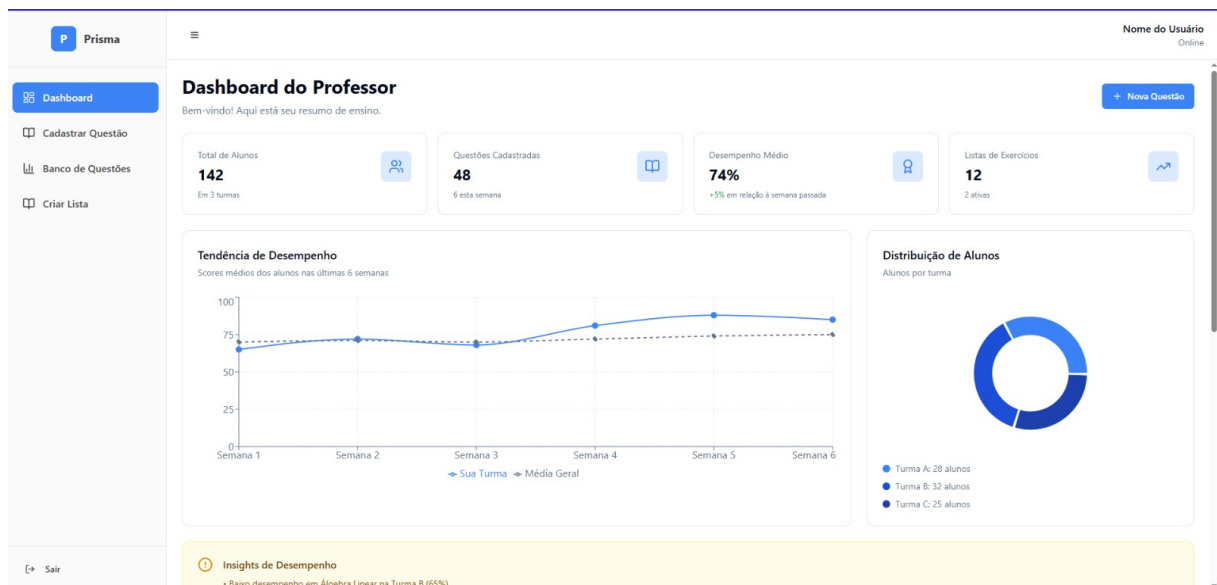


Figura 3: Dashboard do professor.

4.4.3 Tela de Cadastro de Questões

Objetivo: Permitir a inserção de novas questões no banco de dados, com campos para enunciado, alternativas, seleção de prova de origem e assistência de IA para classificação.

Dinâmica de Navegação: Acessada pelo menu lateral, conforme a Figura 4. Após preencher os dados e confirmar, a tela salva a questão e limpa o formulário para um novo cadastro.

Requisitos Funcionais Atendidos: RF01, RF02, RF03.

Cadastrar Questão
Crie novas questões com assistência de IA

Enunciado
Digite o enunciado da questão...

Alternativas

- A Alternativa A
- B Alternativa B
- C Alternativa C
- D Alternativa D
- E Alternativa E

Imagem (opcional)
Clique ou arraste uma imagem
PNG, JPG até 10MB

Preencha o formulário e gere sugestões com IA

Figura 4: Tela de cadastro de questões.

4.4.4 Tela de Criar Listas

Objetivo: Agrupar questões previamente cadastradas e transformá-las em exercícios direcionados a grupos específicos de alunos.

Dinâmica de Navegação: Acessada pelo menu lateral, apresentada na Figura 5. O professor preenche o nome da lista, seleciona a turma e visualiza as questões adicionadas. Pode navegar ao banco de questões para adicionar mais itens sem perder o rascunho.

Requisitos Funcionais Atendidos: RF06, RF07.

Nome da Lista
Ex: Revisão Equações Quadráticas

Selecione a Turma

- ☐ Turma A - Matemática
28 alunos
- ☐ Turma B - Física
32 alunos
- ☐ Turma C - Química
25 alunos

Questões Selecionadas (3)

- 1 Resolver a equação quadrática $x^2 - 5x + 6 = 0$
Matemática
- 2 Calcular a área de um triângulo retângulo
Matemática

Gerar Lista

Adicionar Mais Questões

Dica
Você pode adicionar questões do banco de questões antes de gerar a lista. Certifique-se de selecionar pelo menos uma questão.

Figura 5: Tela de criar listas.

4.4.5 Tela do Banco de Questões

Objetivo: Funcionar como um repositório pesquisável e filtrável de todo o acervo da instituição.

Dinâmica de Navegação: Acessada pelo menu lateral do perfil professor, conforme a Figura 6. Serve como tela de consulta e também atua em conjunto com a tela de Criar Lista.

Requisitos Funcionais Atendidos: RF04.

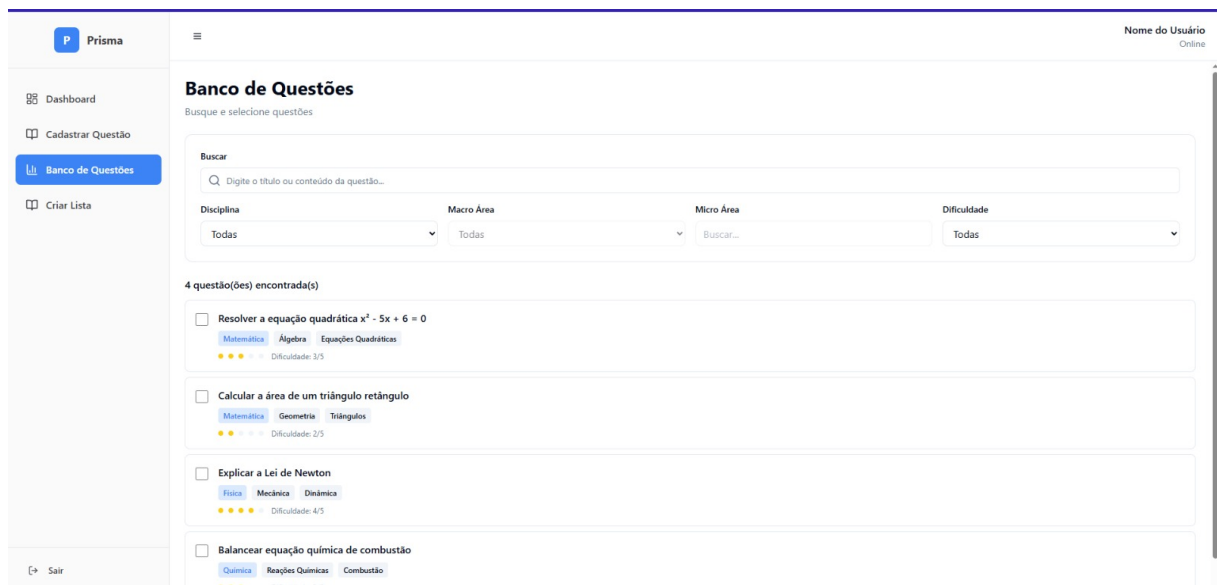


Figura 6: Tela do banco de questões.

4.4.6 Tela de Gerenciamento de Frequência

Objetivo: Fornecer uma interface para que o professor realize a chamada dos alunos de uma turma em uma data específica.

Dinâmica de Navegação: Acessada através do menu lateral, conforme a Figura 7. O professor seleciona a turma e a data, marca o status individual de cada aluno e pode alternar para a aba Histórico para consultar chamadas anteriores.

Requisitos Funcionais Atendidos: RF12.

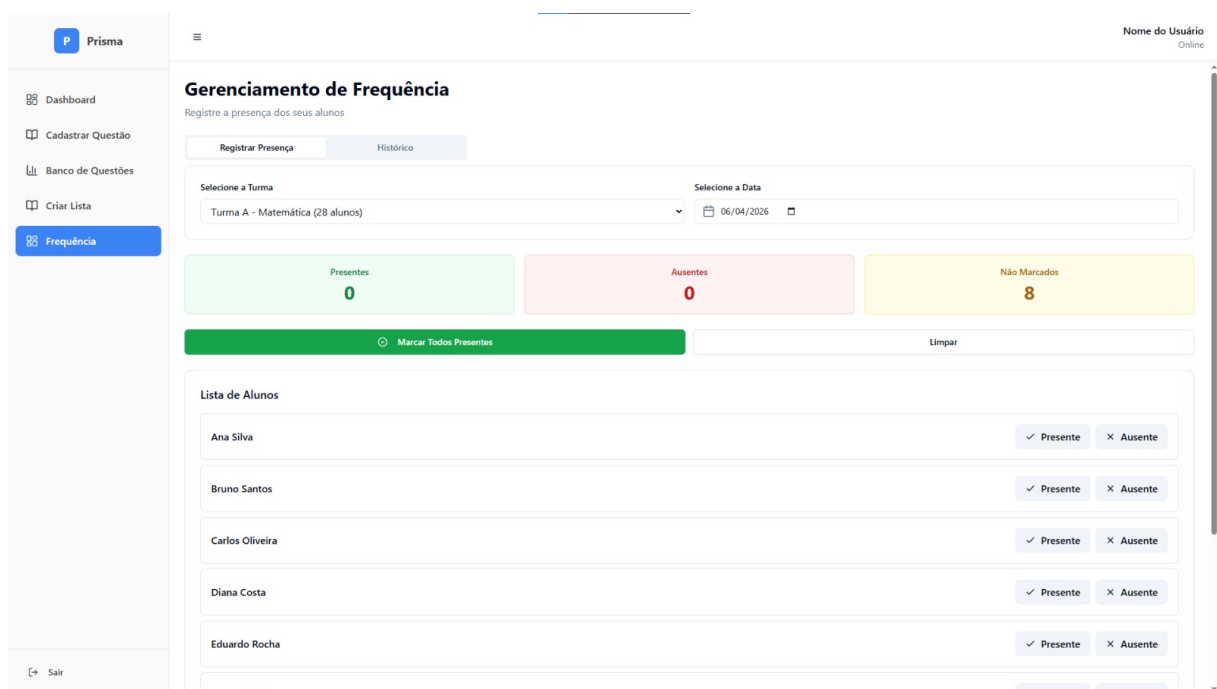


Figura 7: Tela de gerenciamento de frequência.

4.4.7 Dashboard do Aluno

Objetivo: Apresentar a área de estudos e autoconhecimento do aluno, com média geral, questões respondidas, frequência e desempenho por disciplina.

Dinâmica de Navegação: Tela inicial do aluno após o login, conforme a Figura 8. A partir das listas disponíveis, o aluno acessa diretamente a resolução.

Requisitos Funcionais Atendidos: RF09, RF13.



Figura 8: Dashboard do aluno.

4.4.8 Tela de Resolução do Exercício

Objetivo: Ambiente de estudo focado na leitura do enunciado e seleção da alternativa correta, com indicativos de progresso.

Dinâmica de Navegação: Chamada a partir do dashboard do aluno, conforme a Figura 9. O usuário navega com os botões Anterior e Próxima. Ao finalizar, é redirecionado para a tela de gabarito.

Requisitos Funcionais Atendidos: RF08.

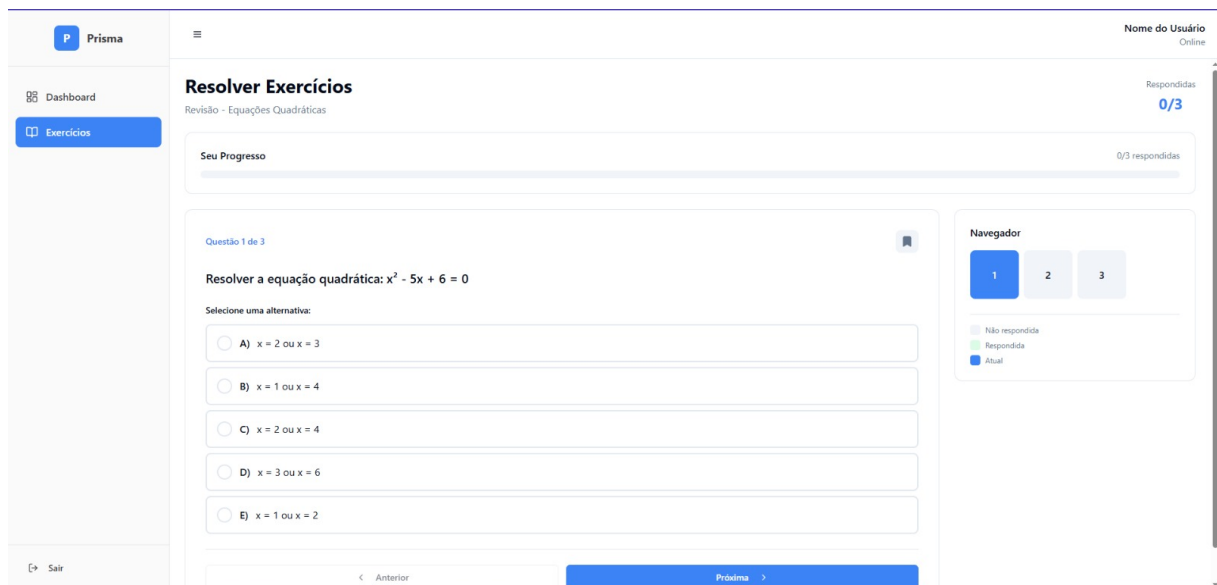


Figura 9: Tela de resolução do exercício.

4.4.9 Painel Administrativo

Objetivo: Permitir que a coordenação gerencie os acessos ao sistema e visualize os logs de atividade.

Dinâmica de Navegação: Tela inicial acessada após o login com o perfil de administrador, conforme a Figura 10. O usuário alterna entre as abas Usuários e Logs.

Requisitos Funcionais Atendidos: RF11.

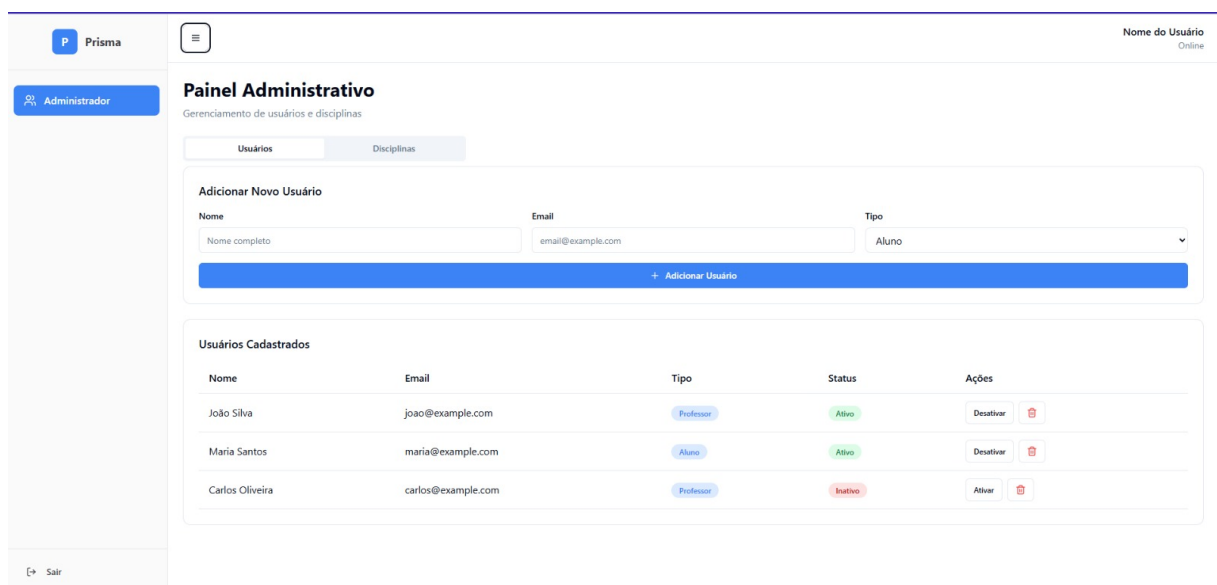


Figura 10: Painel administrativo.

5 Análise do Sistema

5.1 Modelo do Banco de Dados

A modelagem de dados foi projetada para garantir a integridade entre as entidades do Cursinho Prisma. A estrutura lógica do banco de dados relacional está apresentada na Figura 11.

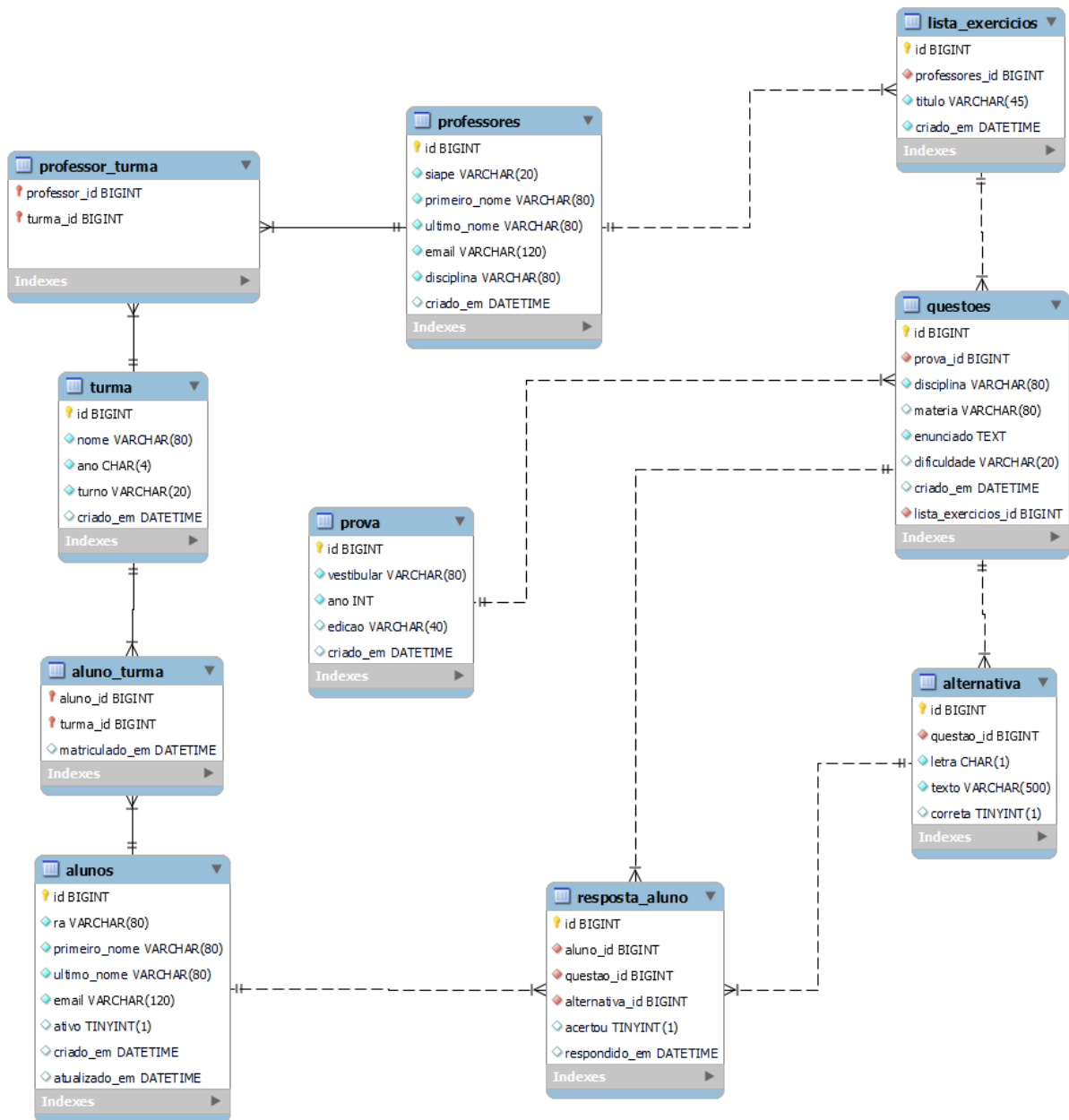


Figura 11: Modelo Entidade-Relacionamento do sistema Prisma, detalhando as chaves primárias e estrangeiras.

Dicionário de Dados:

5.1.1 Dicionário de Dados

Tabela 4: Tabela: professores

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único do professor
siape	VARCHAR(20)	–	Sim	Número do SIAPE do professor
primeiro_nome	VARCHAR(80)	–	Sim	Primeiro nome do professor
ultimo_nome	VARCHAR(80)	–	Sim	Sobrenome do professor
email	VARCHAR(120)	–	Sim	Endereço de e-mail
disciplina	VARCHAR(80)	–	Sim	Nome da disciplina que leciona
criado_em	DATETIME	–	Sim	Data e hora da criação do registro

Tabela 5: Tabela: turma

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único da turma
nome	VARCHAR(80)	–	Sim	Nome descritivo da turma
ano	CHAR(4)	–	Sim	Ano letivo da turma
turno	VARCHAR(20)	–	Sim	Turno das aulas
principal	BOOLEAN	–	Sim	Indica se é a turma principal para cálculo de frequência
criado_em	DATETIME	–	Sim	Data e hora da criação do registro

Tabela 6: Tabela: alunos

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único do aluno
ra	VARCHAR(80)	–	Sim	Registro Acadêmico do aluno
primeiro_nome	VARCHAR(80)	–	Sim	Primeiro nome do aluno
ultimo_nome	VARCHAR(80)	–	Sim	Sobrenome do aluno
email	VARCHAR(120)	–	Sim	Endereço de e-mail do aluno
ativo	TINYINT(1)	–	Sim	Indica se o aluno está ativo (1) ou inativo (0)
criado_em	DATETIME	–	Sim	Data e hora da criação do registro
atualizado_em	DATETIME	–	Não	Data e hora da última atualização

Tabela 7: Tabela: questoes

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único da questão
prova_id	BIGINT	FK	Não	Prova de origem (opcional – questão pode ser avulsa)
disciplina	VARCHAR(80)	–	Sim	Macro-área associada à questão
materia	VARCHAR(80)	–	Sim	Micro-área da questão
enunciado	TEXT	–	Sim	Texto completo da pergunta
dificuldade	VARCHAR(20)	–	Sim	Nível de dificuldade da questão
numero	INT	–	Sim	Número da questão na prova de origem
criado_em	DATETIME	–	Sim	Data e hora da inserção da questão

Tabela 8: Tabela: frequencia

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único do registro
turma_id	BIGINT	FK	Sim	Turma da chamada
aluno_id	BIGINT	FK	Sim	Aluno registrado
data	DATE	–	Sim	Data da aula
presente	BOOLEAN	–	Sim	Indica se o aluno estava presente
criado_em	DATETIME	–	Sim	Data e hora da criação do registro

Tabela 9: Tabela: resposta_aluno

Atributo	Tipo	Chave	Obrig.	Descrição
id	BIGINT	PK	Sim	Identificador único da resposta
aluno_id	BIGINT	FK	Sim	Aluno que respondeu a questão
questao_id	BIGINT	FK	Sim	Questão respondida
alternativa_id	BIGINT	FK	Sim	Alternativa escolhida
acertou	TINYINT(1)	–	Sim	Indica se a resposta está correta
respondido_em	DATETIME	–	Sim	Data e hora em que a resposta foi enviada

5.2 Diagrama de Classes

A estrutura lógica do sistema e as associações entre os módulos estão evidenciadas na Figura 12.

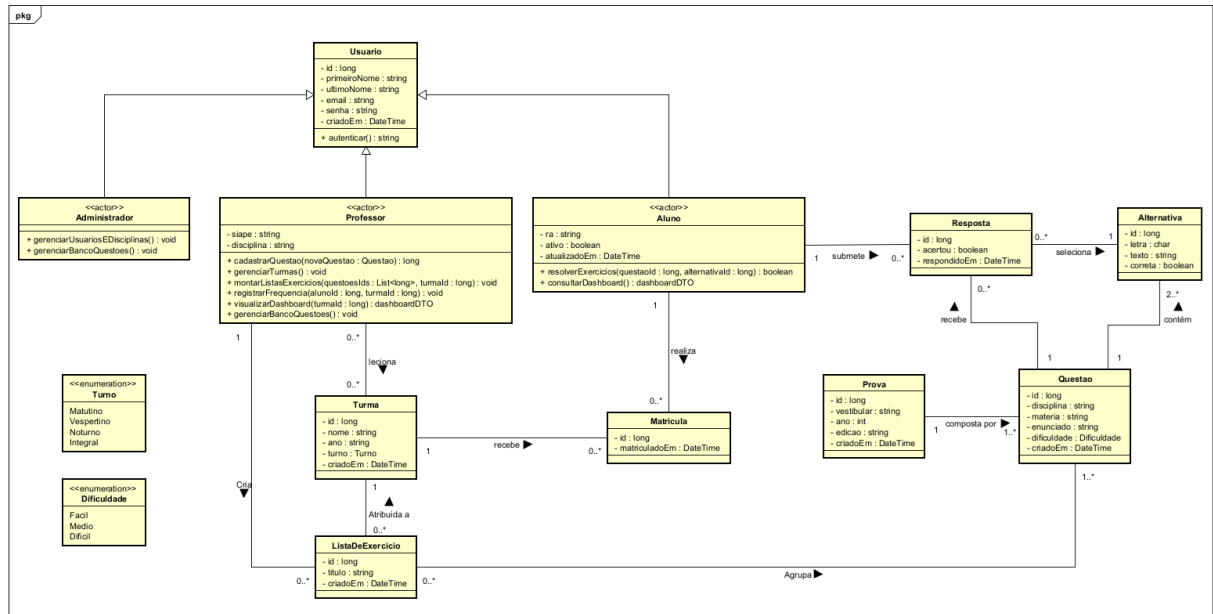


Figura 12: Diagrama de Classes detalhando a estrutura lógica das entidades e as associações entre os módulos do sistema.

5.3 Diagramas de Atividades

O fluxo de cadastro de questões, que integra a sugestão da IA com a validação humana, é detalhado na Figura 13.

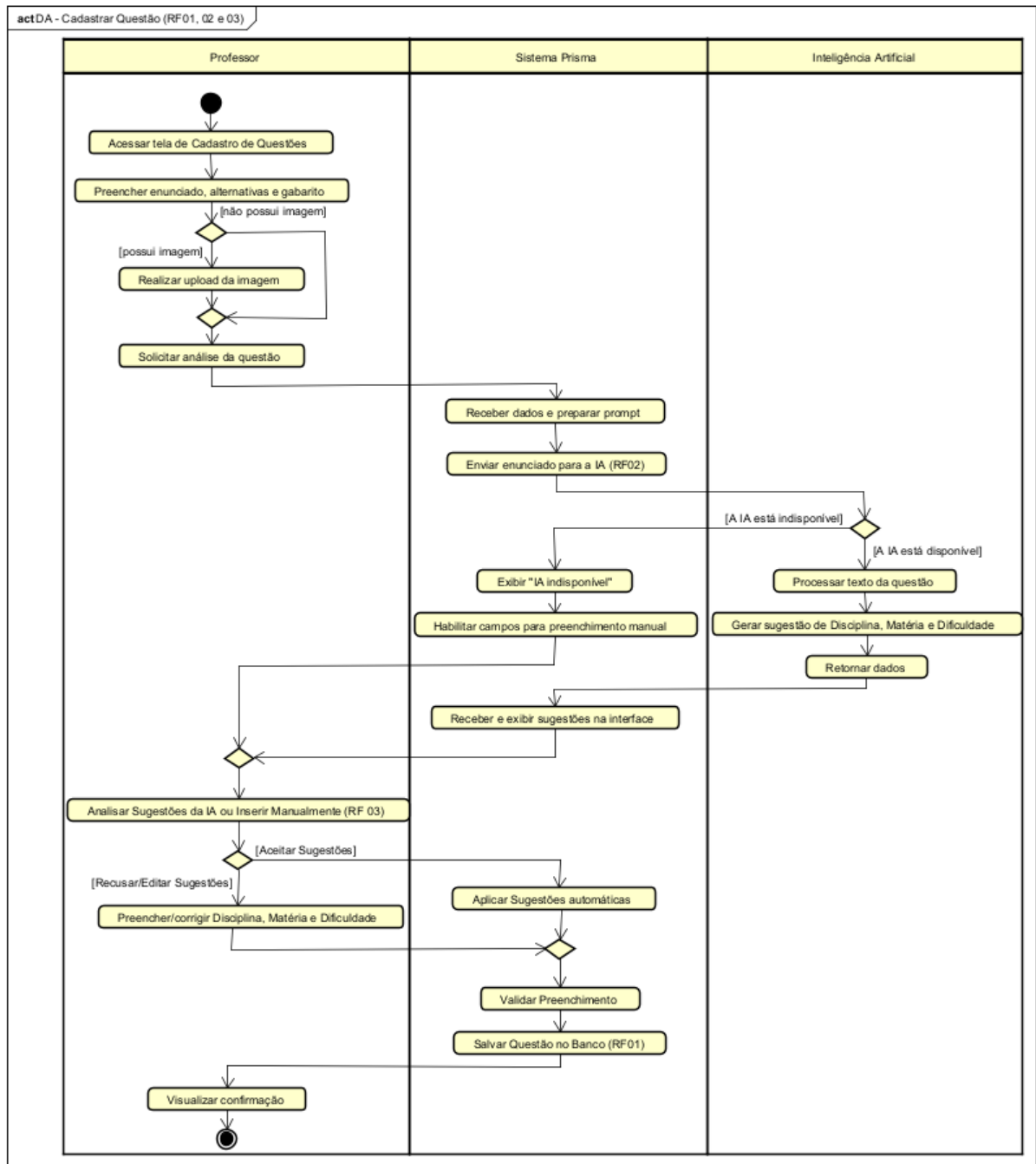


Figura 13: Fluxo de cadastro de questões assistido pela IA – RF01, RF02 e RF03.

A operação de manutenção do banco de questões está ilustrada na Figura 14.

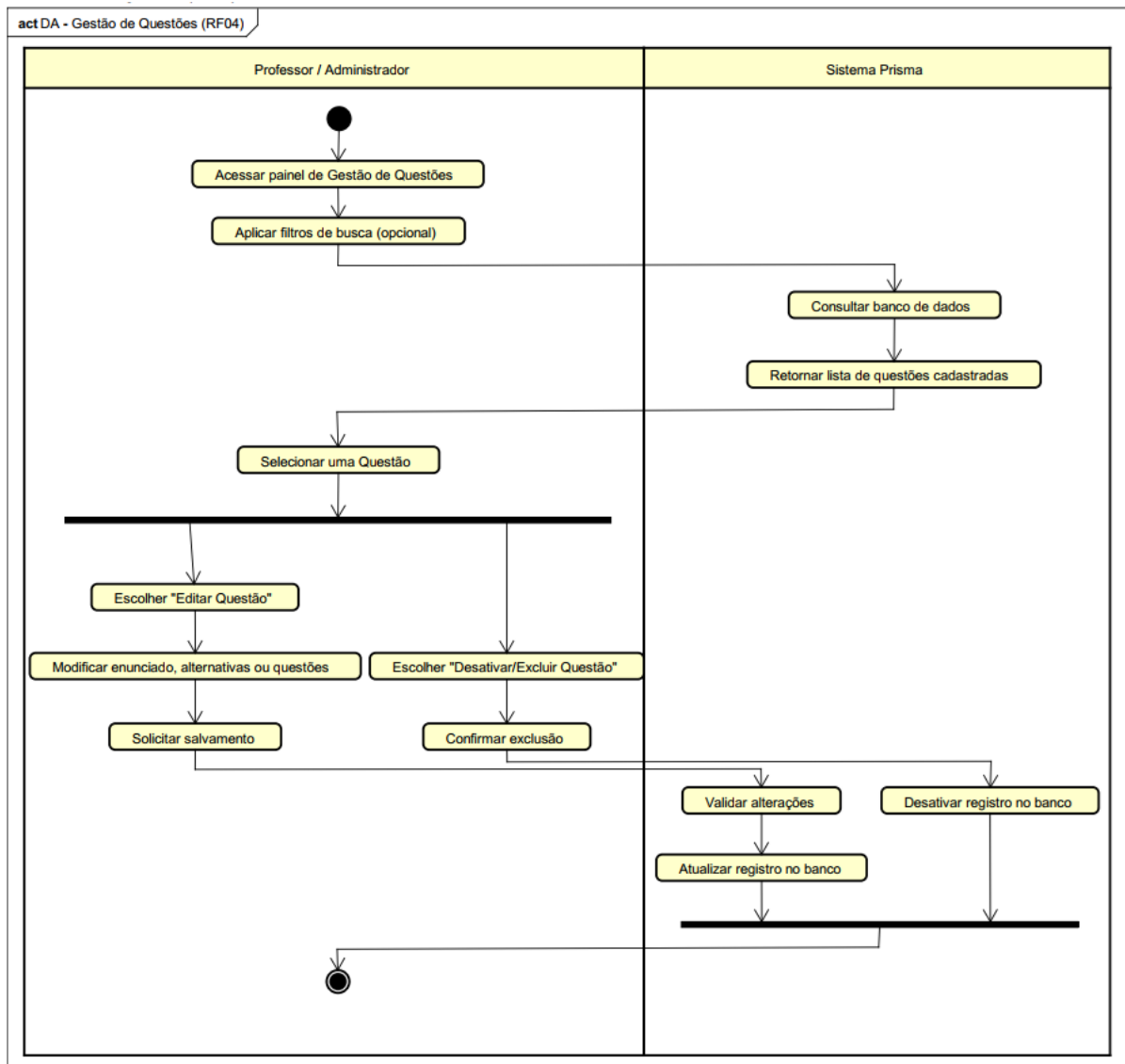


Figura 14: Fluxo de gestão de questões (CRUD) – RF04.

A gestão de turmas está definida na Figura 15.

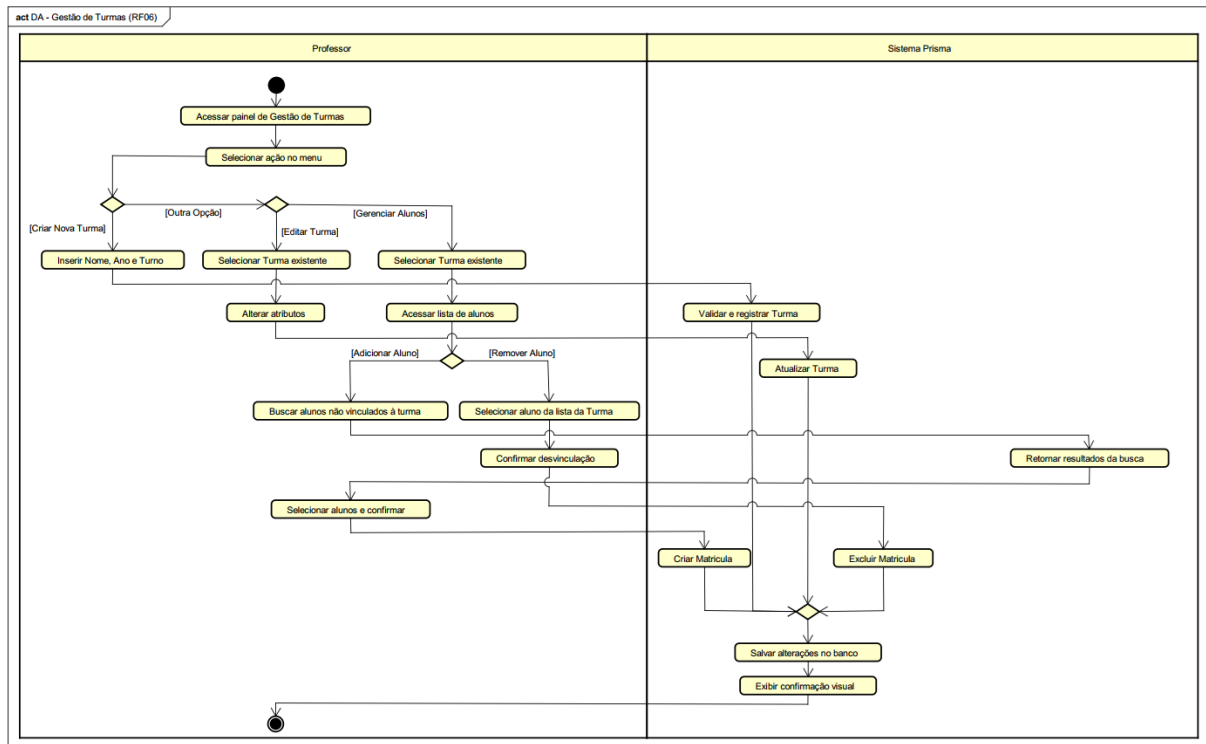


Figura 15: Fluxo de gestão de turmas – RF06.

O procedimento para montagem de listas de exercícios está descrito na Figura 16.

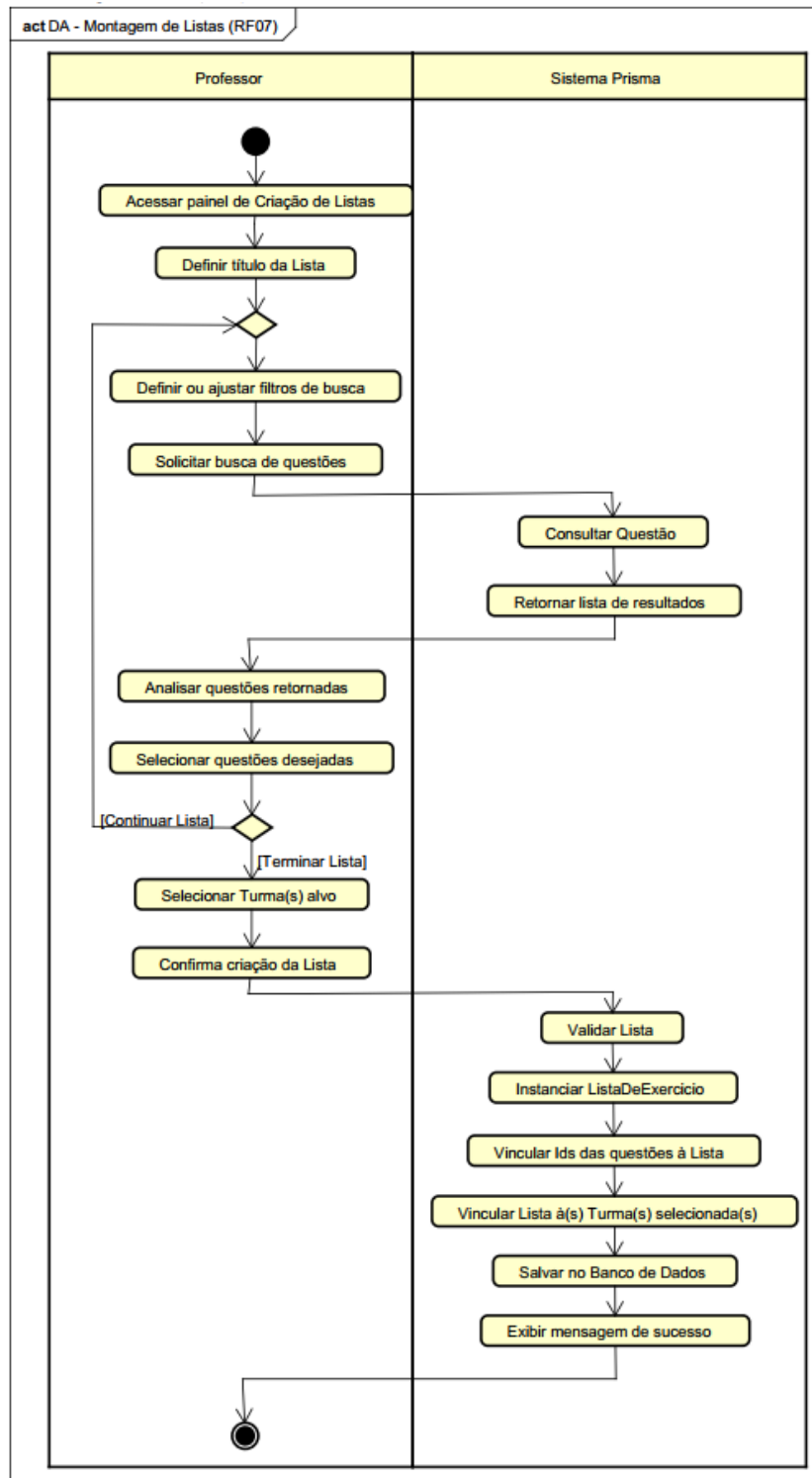


Figura 16: Fluxo de montagem de listas de exercícios – RF07.

O processo de resolução de exercícios pelos alunos está demonstrado na Figura 17.

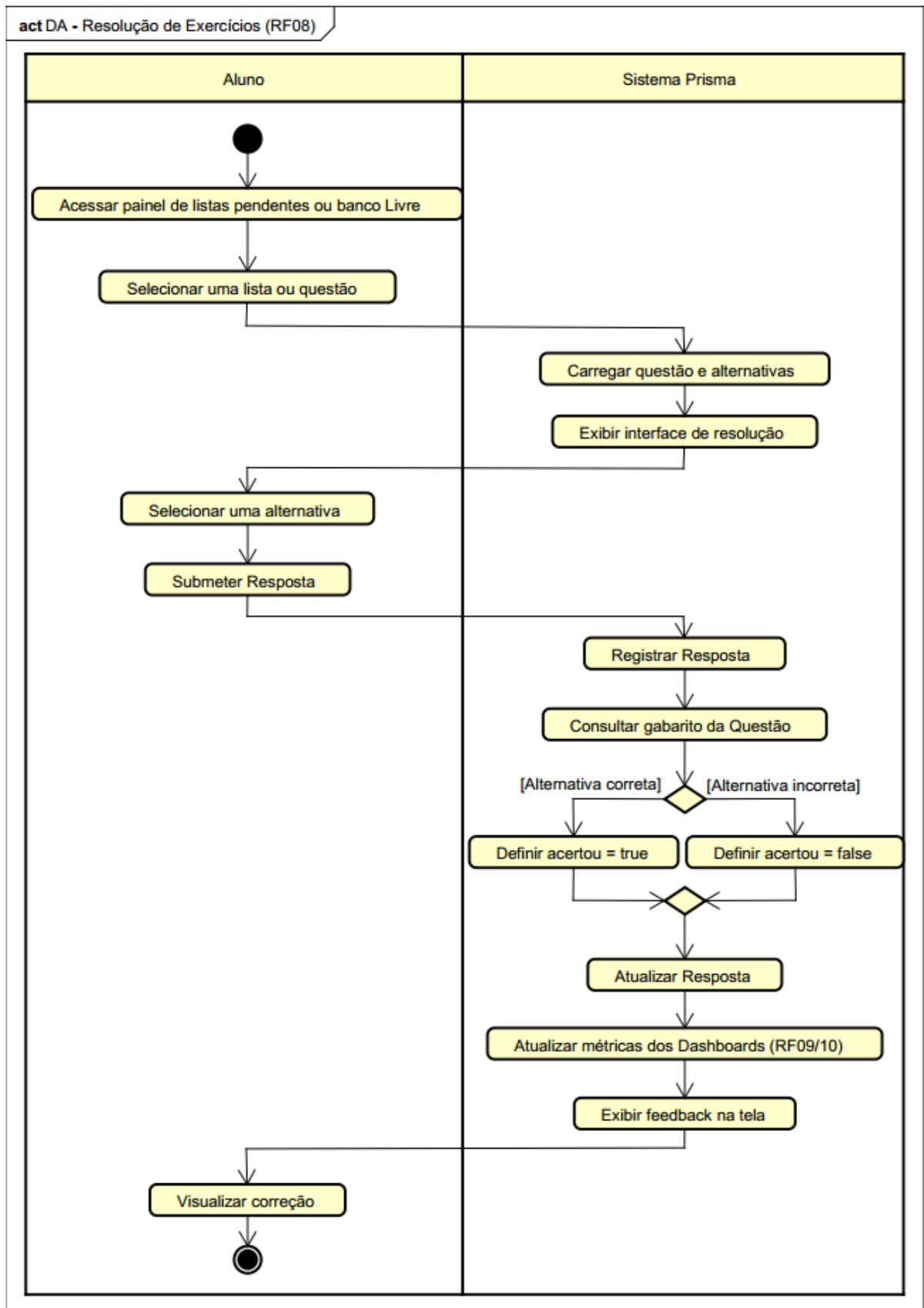


Figura 17: Fluxo de resolução de exercícios – RF08.

A visualização de dados de desempenho pelo aluno está ilustrada na Figura 18.

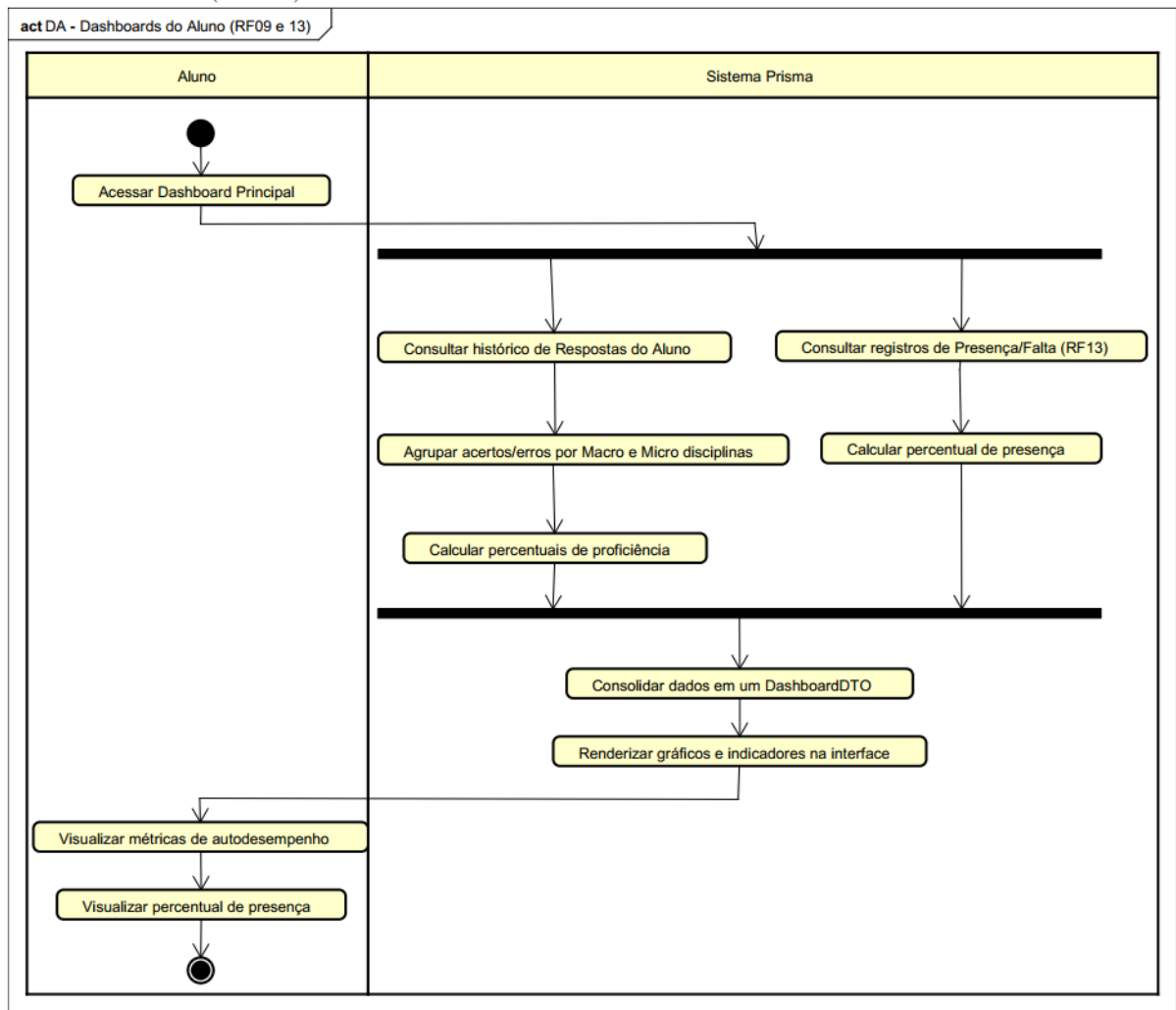


Figura 18: Fluxo de acompanhamento do dashboard do aluno – RF09 e RF13.

A análise de performance da turma pelo professor está detalhada na Figura 19.

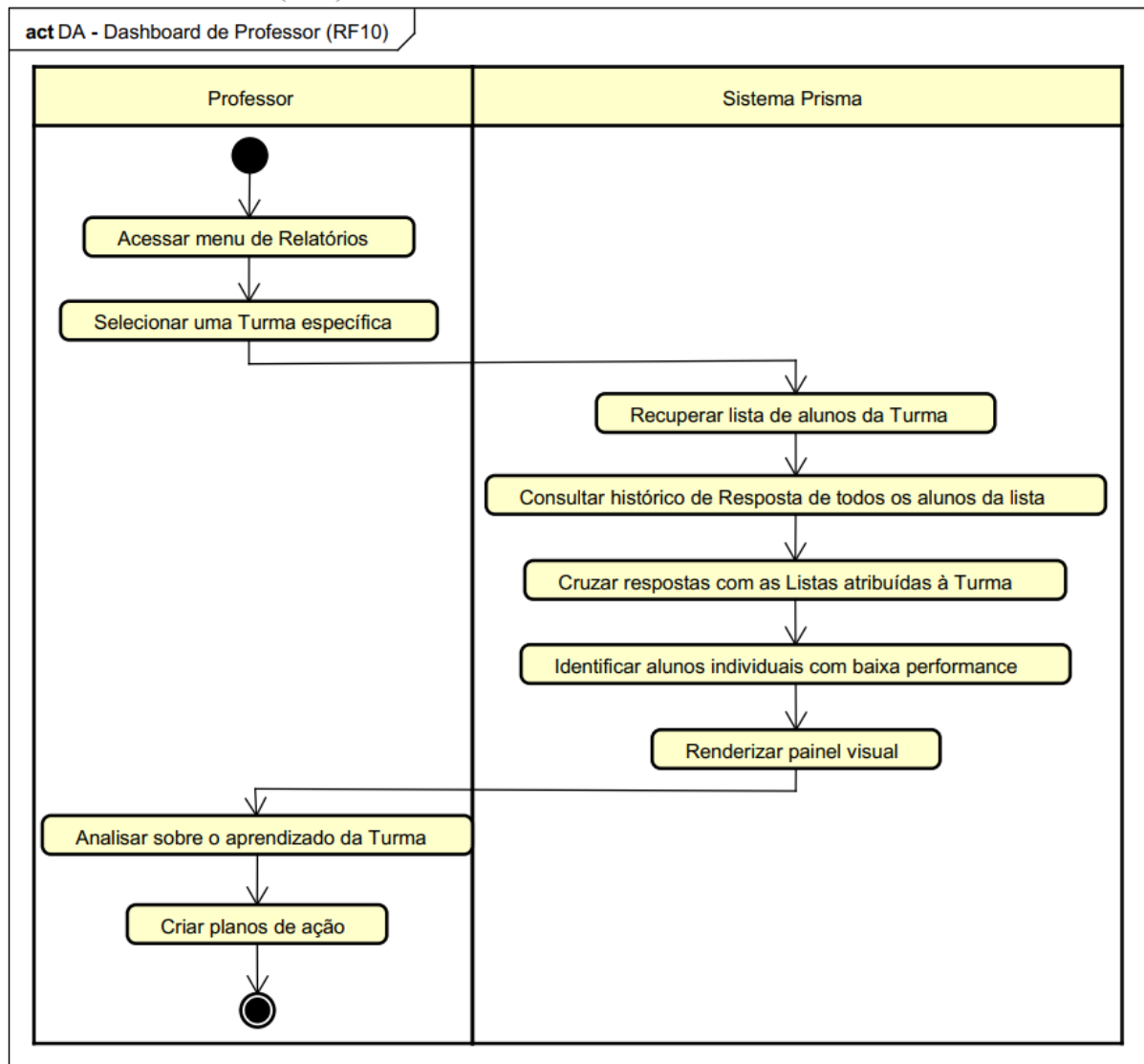


Figura 19: Fluxo de visualização do dashboard do professor – RF10.

A gestão administrativa de usuários está apresentada na Figura 20.

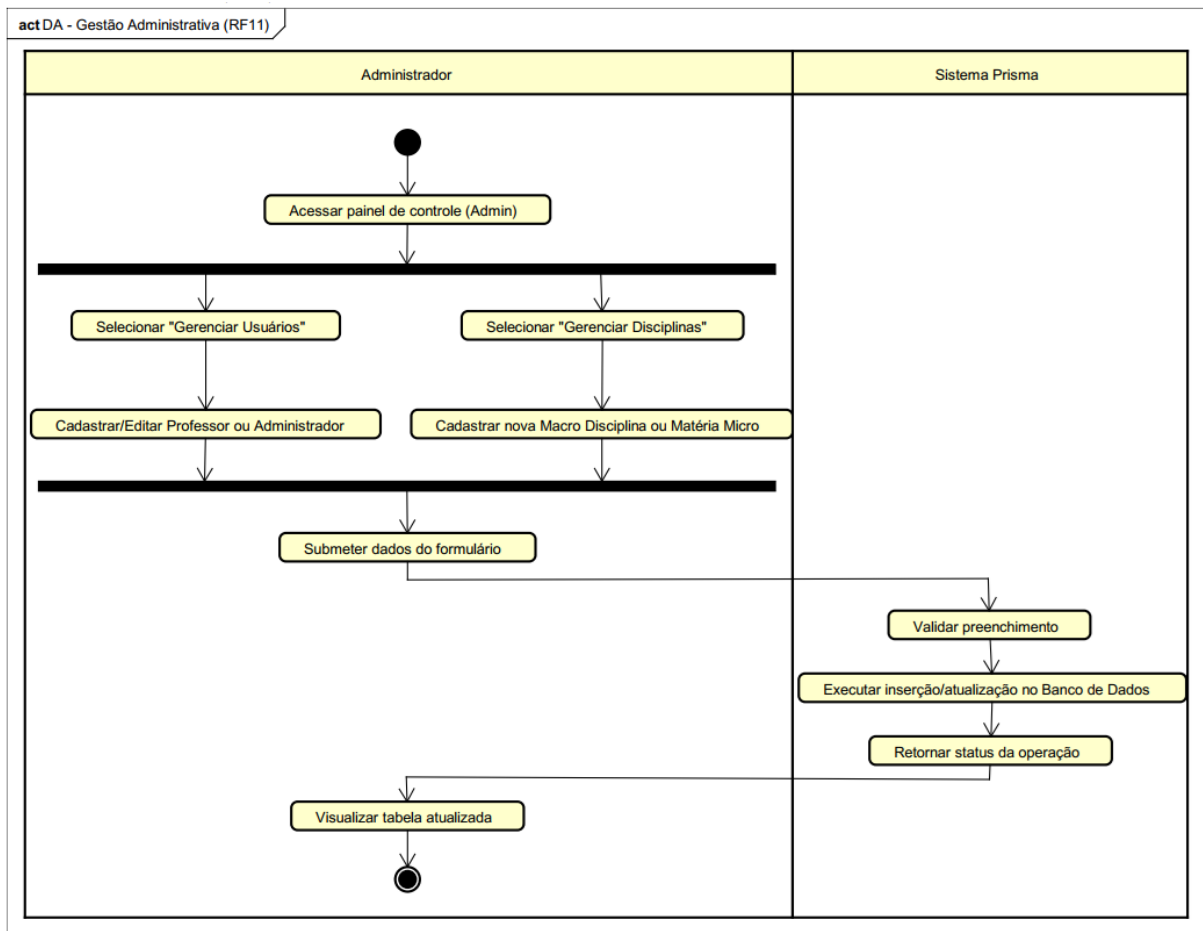


Figura 20: Fluxo da gestão administrativa – RF11.

O registro de frequência escolar está descrito na Figura 21.

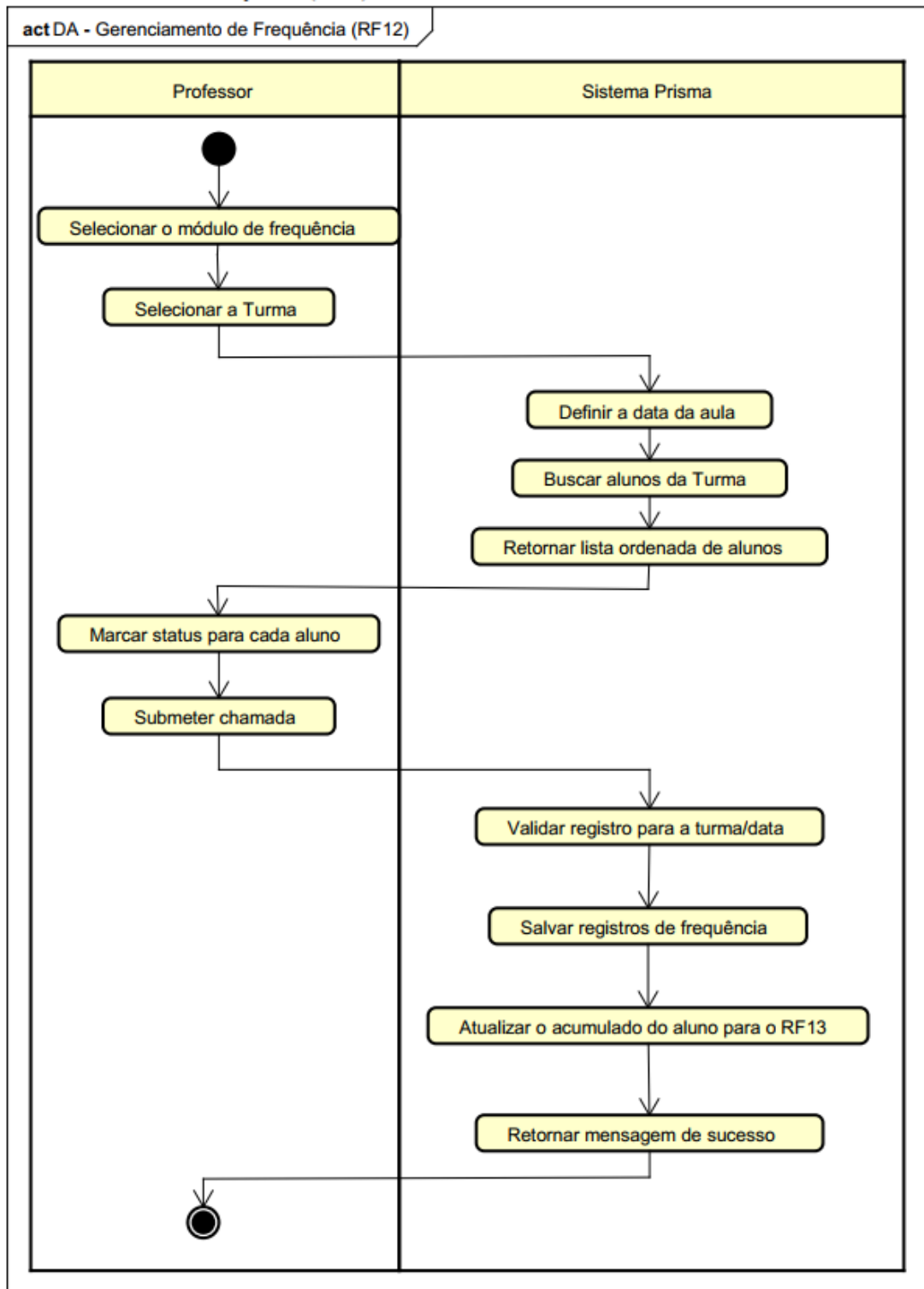


Figura 21: Fluxo de gerenciamento de frequência – RF12.

6 Implementação

6.1 Descrição do código

O sistema foi desenvolvido com uma arquitetura de microsserviços no back-end, uma aplicação single page application (SPA) no front-end e um serviço isolado para a inteligência artificial. O repositório é organizado como um monorepo, dividido em três diretórios principais: `backend/`, `frontend/` e `ia/`. A Figura 22 apresenta o diagrama de componentes do sistema.

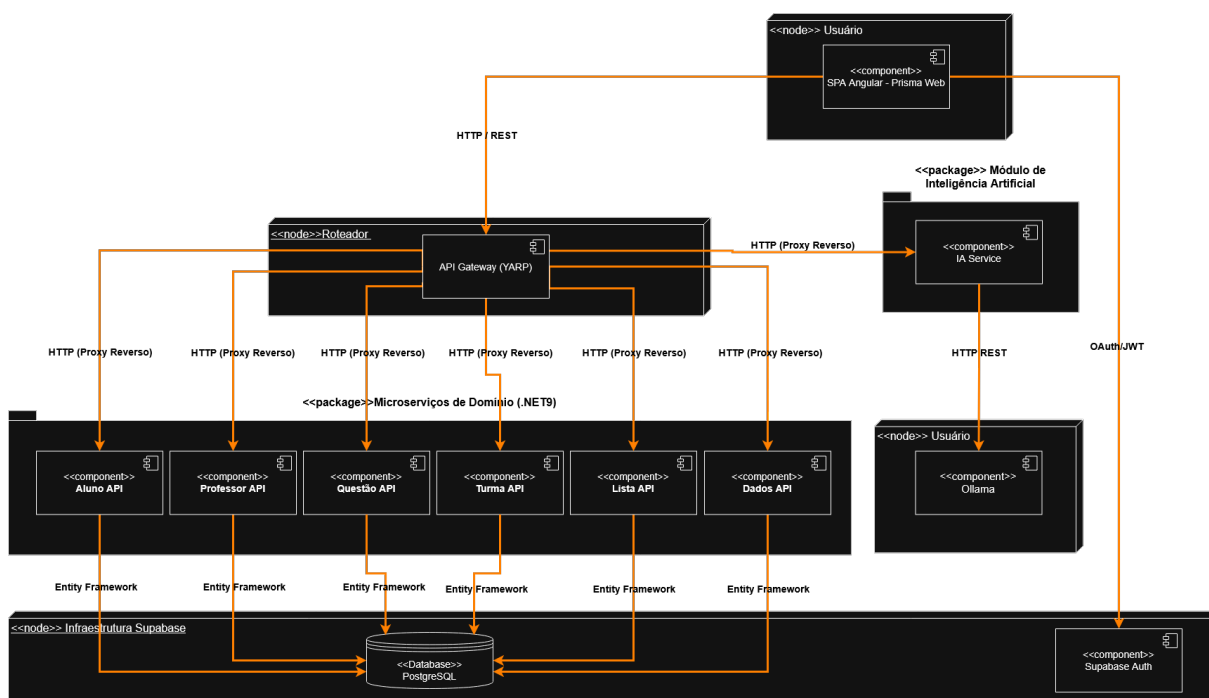


Figura 22: Diagrama de componentes do sistema Prisma.

6.1.1 Back-End (.NET 9 / C#)

O back-end é composto por um API Gateway construído com YARP, que recebe todas as requisições do front-end e as repassa para o microsserviço correto, e por seis microsserviços de domínio: `AlunoMicroservice`, `ProfessorMicroservice`, `QuestaoMicroservice`, `TurmaMicroservice`, `ListaMicroservice` e `DadosMicroservice`.

Cada microsserviço adota uma arquitetura em camadas com os seguintes pacotes:

- **Controllers:** exposição dos endpoints REST;
- **Services:** regras de negócio;
- **Repositories:** isolamento do acesso ao banco de dados;
- **DTOs:** objetos de transferência de dados, que evitam expor as entidades diretamente;

- **Model:** entidades mapeadas para o banco PostgreSQL via Entity Framework;
- **Mappers:** conversores entre Models e DTOs.

A autenticação é feita via JWT emitido pelo Supabase Auth. O `ProfessorMicroservice` concentra também o `AdminController`, responsável por criar usuários diretamente na autenticação do Supabase e inserir o registro correspondente no banco.

6.1.2 Front-End (Angular 21+)

O front-end utiliza standalone components e segue uma estrutura baseada em funcionalidades:

- `src/app/core/`: contém os serviços globais, interceptors de autenticação, guards de rota e os layouts base (professor, aluno e admin);
- `src/app/features/`: divide o sistema por domínios de negócio isolados — `admin`, `aluno`, `auth` e `professor`.

O `AuthService` gerencia o login via SDK do Supabase, armazena o token JWT no `localStorage` e expõe o perfil do usuário logado para os demais componentes.

6.1.3 Inteligência Artificial (Python / FastAPI / Ollama llama3.2:3b)

O serviço de IA é uma API REST independente escrita em Python com FastAPI. Ele roda localmente na máquina host, fora dos containers Docker, e se comunica com o Ollama pelo endereço `host.docker.internal:11434`. Os arquivos principais são:

- `main.py`: ponto de entrada da API;
- `classifier.py`: integração com o Ollama para classificação de enunciados;
- `prompt_builder.py`: construção dos prompts com exemplos de calibração por disciplina e nível de dificuldade para todas as 9 disciplinas (matemática, química, física, biologia, história, geografia, linguagens, filosofia e sociologia);
- `validator.py`: validação e normalização das respostas do modelo;
- `schemas.py`: definição dos contratos de dados da API.

O serviço recebe o enunciado de uma questão e retorna sugestões de disciplina (macro-área), matéria (micro-área) e nível de dificuldade. O professor pode aceitar, editar ou recusar cada sugestão antes de salvar a questão.

6.2 Implantação

Esta seção descreve os requisitos e os passos necessários para executar o sistema em ambiente local.

6.2.1 Requisitos de Software

Tabela 10: Ferramentas necessárias para implantação

Ferramenta	Versão mínima	Finalidade
Git	qualquer	Clonar o repositório
Docker Desktop	4.x	Executar os microserviços em containers
Node.js	20 LTS	Executar o front-end Angular
Angular CLI	21.x	Servir o front-end localmente
Ollama	qualquer	Executar o modelo de linguagem local

O .NET SDK e o Python não precisam ser instalados na máquina, pois são executados dentro dos containers Docker. O Angular CLI pode ser instalado após o Node.js com o comando:

```
1 npm install -g @angular/cli
```

6.2.2 Passo 1 — Instalar e configurar o Ollama

O Ollama precisa estar rodando antes de subir o Docker. Após instalar pelo site oficial (<https://ollama.com/download>), execute no cmd:

```
1 ollama pull llama3.2:3b
2 ollama list
```

O modelo llama3.2:3b deve aparecer na lista. Se o Ollama não estiver rodando, o botão “Sugerir IA” retornará erro, mas o restante do sistema funcionará normalmente.

6.2.3 Passo 2 — Clonar o repositório

```
1 git clone https://github.com/pukanski/OficinaDeIntegracao1.
   git
2 cd OficinaDeIntegracao1
```

6.2.4 Passo 3 — Configurar as variáveis de ambiente

O sistema já vem com os arquivos de configuração preenchidos. Não é necessário criar contas ou configurar serviços externos.

Arquivo .env (back-end) — Localização: OficinaDeIntegracao1/.env

```

1      SUPABASE_URL=https://rjygpcapmeyffuttdgjr.supabase.co
2      SUPABASE_SERVICE_KEY=sb_secret_yJawUT49r4IEm-
        Lqg_TpAg_F_BJs7MT
3      DATABASE_URL=Host=aws-1-sa-east-1.pooler.supabase.com;Port
        =6543;
4      Database=postgres;Username=postgres.rjygpcapmeyffuttdgjr;
5      Password=z3mmvG5gJ9tM402M;Ssl Mode=Require;Pooling=false;

```

Arquivo environment.ts (front-end) —

Localização: frontend/prisma-web/src/environments/environment.ts

```

1      export const environment = {
2          production: false,
3          gatewayUrl: 'http://localhost:5050',
4          supabaseUrl: 'https://rjygpcapmeyffuttdgjr.supabase
        .co',
5          supabaseAnonKey: 'sb_publishable_qlncldtta-4
        YouvRPTz60A_wWtfheJj'
6      };

```

6.2.5 Passo 4 — Subir o back-end com Docker

Na raiz do projeto, execute:

```

1      docker-compose up -d --build

```

Na primeira execução o processo pode levar entre 5 e 10 minutos. Os containers iniciados são apresentados na Tabela 11.

Tabela 11: Containers Docker do sistema

Container	Porta	Descrição
api-gateway	5050	Recebe todas as requisições do front-end
aluno-api	interno	Gerencia alunos e registros de frequência
professor-api	interno	Gerencia professores, admin e criação de usuários
questao-api	interno	Gerencia questões, alternativas e provas
turma-api	interno	Gerencia turmas e matrículas
lista-api	interno	Gerencia listas de exercícios
dados-api	interno	Gerencia o histórico de respostas e métricas
ia-service	interno	API Python de classificação, acessa o Ollama do host

Para verificar se todos os containers estão rodando:

```

1      docker ps

```

6.2.6 Passo 5 — Subir o front-end

Em um terminal separado:

```
1      cd frontend/prisma-web
2      npm install
3      ng serve
```

Aguarde a mensagem `Compiled successfully`. O front-end estará disponível em `http://localhost:4200`.

6.2.7 Passo 6 — Acessar o sistema

As contas disponíveis para acesso estão listadas na Tabela 12.

Tabela 12: Credenciais de acesso ao sistema

Perfil	E-mail	Senha	
Administrador	admin@oficina.com	123456	
Professor	bruna.campos@oficina.com	123456	
Aluno	gabriel.melo@oficina.com	123456	

6.2.8 Solução de Problemas Comuns

Tabela 13: Problemas comuns e suas soluções

Problema	Solução
Botão “Sugerir IA” retorna erro	Verificar com <code>ollama list</code> se o modelo está disponível
IA demora na primeira requisição	Aguardar – o modelo está carregando na memória
Erro de conexão com o banco	Verificar se o <code>.env</code> está na raiz do projeto
<code>ng serve</code> falha com erro de módulo	Executar <code>npm install</code> dentro de <code>frontend/prisma-web</code>
Login não redireciona	Verificar se o container <code>api-gateway</code> está rodando na porta 5050

6.3 Telas Principais

6.3.1 Login

A tela inicial do sistema, apresentada na Figura 23, permite que o usuário selecione seu perfil (Professor, Aluno ou Admin), insira as credenciais e seja redirecionado para o dashboard correspondente. Também é possível solicitar a redefinição de senha pelo link “Esqueci a senha”, que aciona o fluxo de e-mail do Supabase.

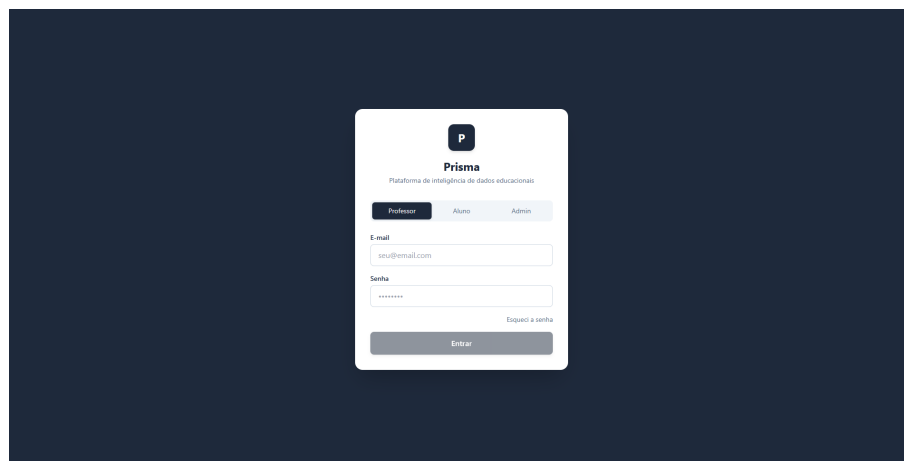


Figura 23: Tela de login com seleção de perfil.

6.3.2 Dashboard do Professor

A Figura 24 apresenta a tela inicial do professor após o login. Exibe quatro indicadores: total de alunos únicos, total de questões no banco, média de desempenho da turma selecionada e total de listas criadas. O professor pode alternar entre suas turmas pelo seletor, o que atualiza o painel de desempenho por disciplina e a lista de alunos com baixo rendimento.

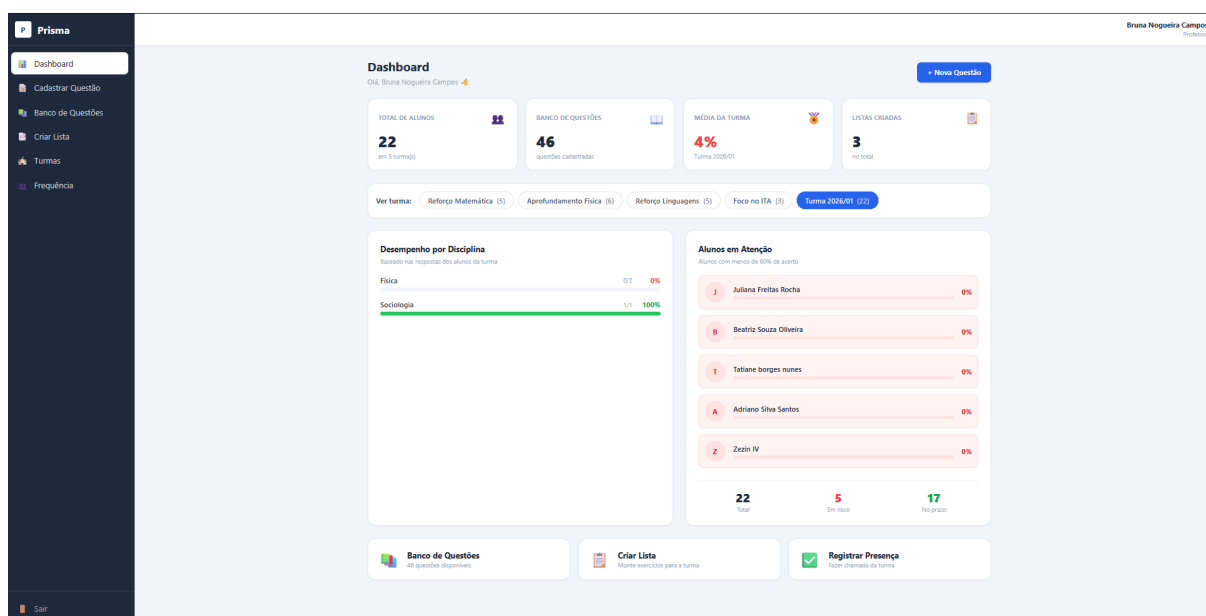


Figura 24: Dashboard do professor com indicadores de desempenho.

6.3.3 Cadastrar Questão

A tela de cadastro, apresentada na Figura 25, permite ao professor preencher o enunciado, as cinco alternativas e marcar o gabarito. O campo de prova de origem usa autocomplete. Ao preencher o enunciado, o botão “Sugerir IA” aciona o serviço de

classificação, que retorna sugestões de disciplina, matéria e dificuldade para revisão do professor.

Prisma
Dashboard
Cadastrar Questão
Banco de Questões
Criar Lista
Turmas
Frequência
Sair

Cadastrar Questão
Crie novas questões com assistência de IA

Prova de Origem + Nova Prova

Buscar prova
Digite vestibular, ano ou edição...
Deixe em branco para questão avulsa.

Número da questão *
1

Enunciado *
Digite o enunciado da questão...

Alternativas e Gabarito *

☒ A Alternativa A
☐ B Alternativa B
☐ C Alternativa C
☐ D Alternativa D
☐ E Alternativa E

Classificação Sugerir IA

Classifique manualmente ou clique em Sugerir IA.

Macro-área (Disciplina) *
Selecione a macro-área

Micro-área (Matéria)
Ex: Cinemática

Dificuldade *
Selecione a dificuldade

Salvar Questão

Figura 25: Tela de cadastro de questão com sugestão da IA.

6.3.4 Banco de Questões

O banco de questões, apresentado na Figura 26, é o repositório de todas as questões cadastradas. Permite filtrar por disciplina, prova, micro-área e dificuldade. Ao selecionar uma questão, a barra de ações aparece com as opções de visualizar, editar ou excluir. Também é possível selecionar múltiplas questões e criar uma lista diretamente.

Prisma
Dashboard
Cadastrar Questão
Banco de Questões
Criar Lista
Turmas
Frequência
Sair

Banco de Questões
Busque e selecione questões

Buscar
Digite o enunciado da questão...

Disciplina: Todas Prova: Buscar prova... Micro Área: Buscar matéria... Dificuldade: Todas

46 questão(ões) encontrada(s)

☐ Pressão, depressão, estresse e crise de ansiedade. Os males da sociedade contemporânea também estão no esporte. A tenista Naomi Osaka, do Japão, jogadora mais bem paga do mundo e que já ocupou o número 2 do ranking, retirou-se do torneio de Roland Garros de 2021 porque não estava conseguindo administrar as crises de ansiedade provocadas pelos grandes eventos, por ser uma estrela aos 23 anos, e pelo peso de parte da...

☒ Em um experimento realizado para determinar a temperatura de equilíbrio térmico entre dois corpos, um pesquisador utiliza um termômetro calibrado na escala Celsius. Ele sabe que a relação entre a temperatura na escala Celsius (TC) e a temperatura na escala Kelvin (TK) é dada por $TK = TC + 273$. Se o termômetro indicar uma temperatura de 25 °C, qual será o valor correspondente dessa temperatura na escala Kelvin?

Física Termometria ENEM 2011
Muito Fácil

☐ Uma pessoa verticalmente imóvel diante de um espelho plano vertical, fixo em uma parede, observa a imagem de seu próprio corpo. Para que essa pessoa consiga ver seu corpo inteiro refletido no espelho, desde o topo da cabeça até os pés, o comprimento mínimo vertical desse espelho deve ser igual a:

Física Óptica UNESP 2016
Fácil

1 selecionada(s) Visualizar Editar Excluir Criar Lista

☐ Um veículo estuda o movimento de duas partículas que se movem ao longo de uma mesma linha reta. A partícula A se move com velocidade constante de 20 metros por segundo. A partícula B move-se no mesmo sentido com velocidade constante de 10 metros por segundo. Partícula A está a 100 metros da origem e partícula B está a 50 metros da origem. Qual o tempo necessário para que as duas partículas se encontrem?

Figura 26: Banco de questões com filtros e barra de ações.

6.3.5 Criar Lista

A tela de criação de listas, apresentada na Figura 27, permite ao professor definir o título, uma data de vencimento opcional e selecionar as turmas que receberão a lista. As questões são adicionadas pelo banco — ao navegar para o banco e voltar, as questões marcadas aparecem pré-selecionadas e o rascunho da lista é preservado.

Figura 27: Tela de criação de lista com questões selecionadas.

6.3.6 Gerenciar Turmas

A tela de gerenciamento de turmas, apresentada na Figura 28, permite criar, editar e excluir turmas, além de matricular e desmatricular alunos. Cada turma pode ser marcada como “principal”, o que determina qual chamada é usada no cálculo de frequência dos alunos. A busca por nome ou RA facilita encontrar alunos em turmas com muitos membros.

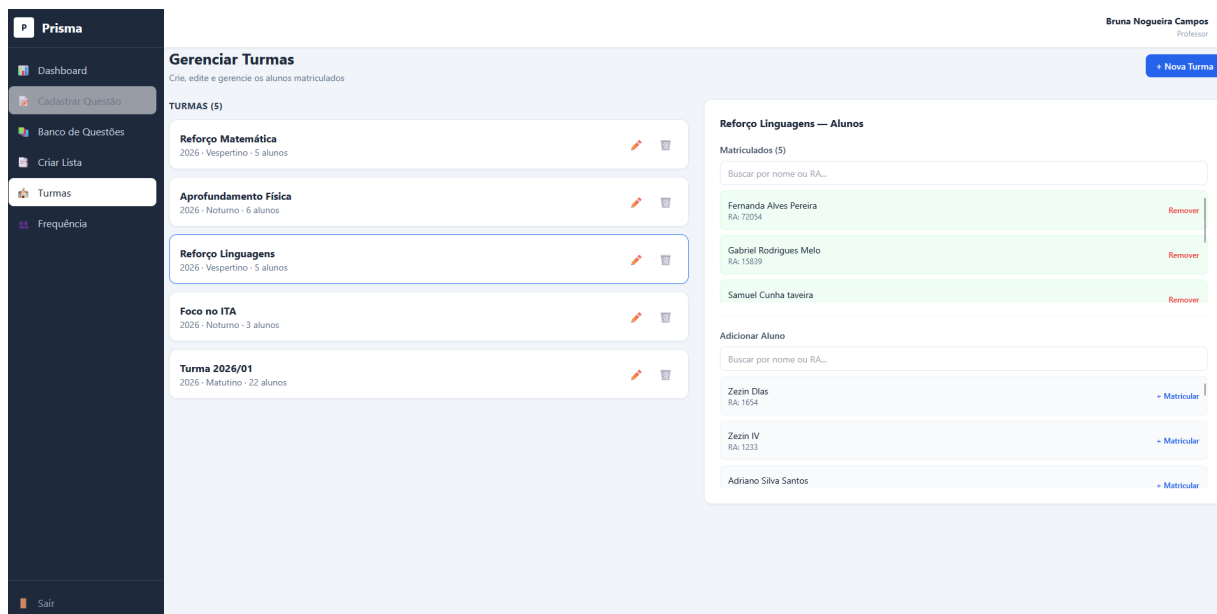


Figura 28: Tela de gerenciamento de turmas com painel de alunos.

6.3.7 Dashboard do Aluno

A Figura 29 apresenta a tela inicial do aluno. Exibe a média geral de acertos, o total de questões respondidas, o percentual de frequência — calculado apenas sobre a turma principal — e a melhor disciplina. O painel de desempenho mostra barras por disciplina com o percentual de acerto. As listas atribuídas pelas turmas aparecem na coluna da direita.

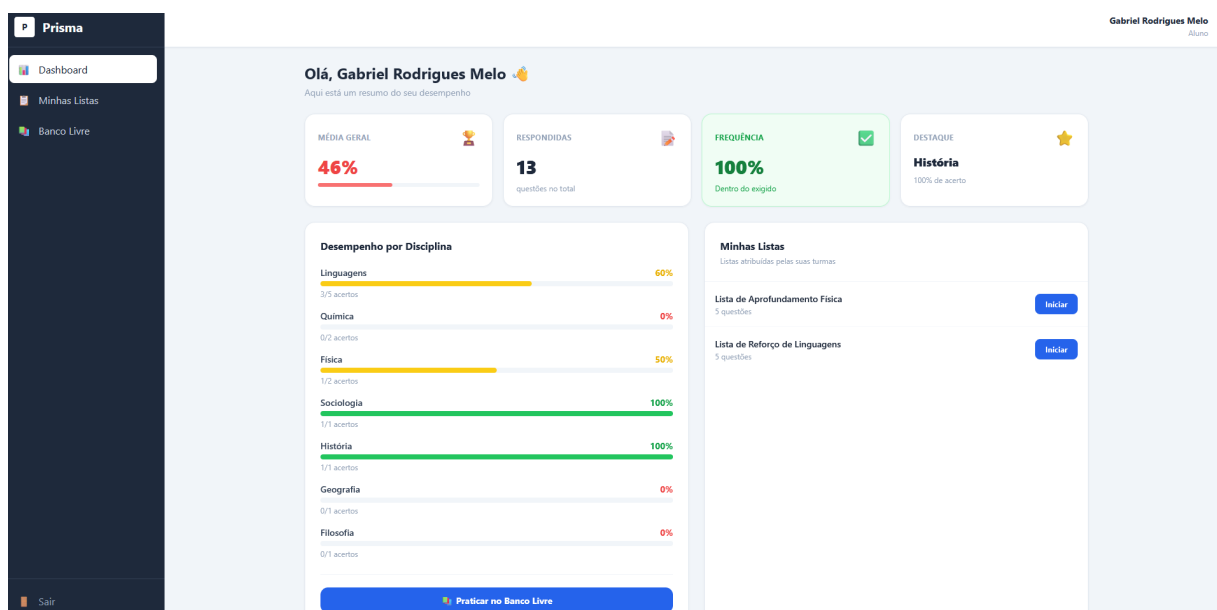


Figura 29: Dashboard do aluno com métricas de desempenho e listas disponíveis.

6.3.8 Resolução de Exercícios

A tela de resolução, apresentada na Figura 30, é o ambiente onde o aluno responde as questões de uma lista. No topo é exibido o nome da lista e o progresso atual. O enunciado e as alternativas ocupam o centro da tela, e o navegador lateral permite visualizar o status de cada questão — atual, respondida ou não respondida — e navegar diretamente para qualquer uma delas. Ao finalizar, o aluno é redirecionado para o gabarito com a correção completa.

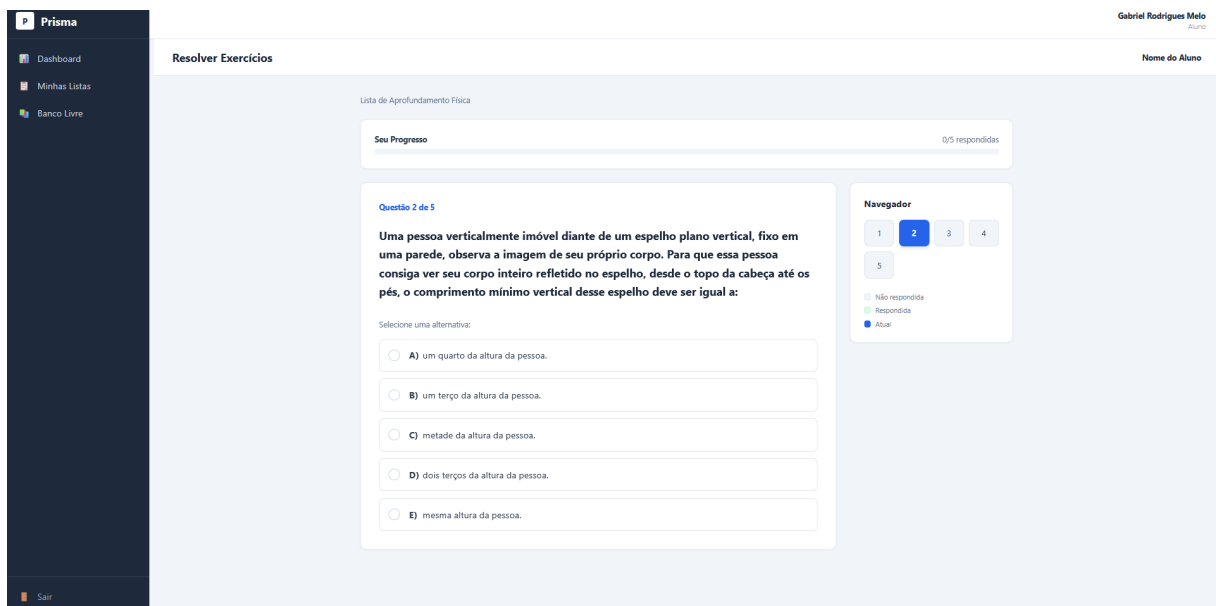


Figura 30: Tela de resolução de exercícios com navegador lateral de questões.

6.3.9 Banco Livre

O banco livre, apresentado na Figura 31, é o repositório de questões acessível ao aluno sem necessidade de lista. O aluno filtra por disciplina, prova, micro-área ou dificuldade e clica em qualquer questão para respondê-la em um modal. O resultado é mostrado com o gabarito destacado, e a resposta é registrada no histórico que alimenta os dashboards.

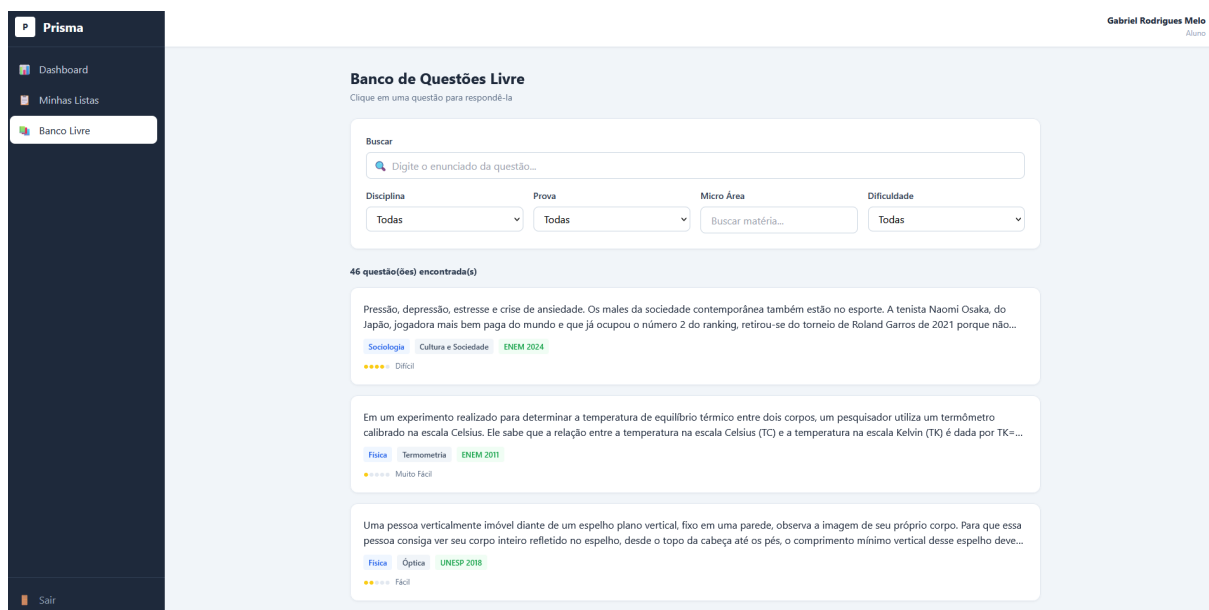


Figura 31: Banco livre do aluno com modal de resolução.

6.3.10 Painel Administrativo

O painel administrativo, apresentado na Figura 32, é a tela exclusiva do administrador. Permite criar contas de professores e alunos em uma única operação. A aba de Logs exibe os registros de criação e atualização de usuários, questões, listas e turmas, com busca por texto e filtro por categoria.

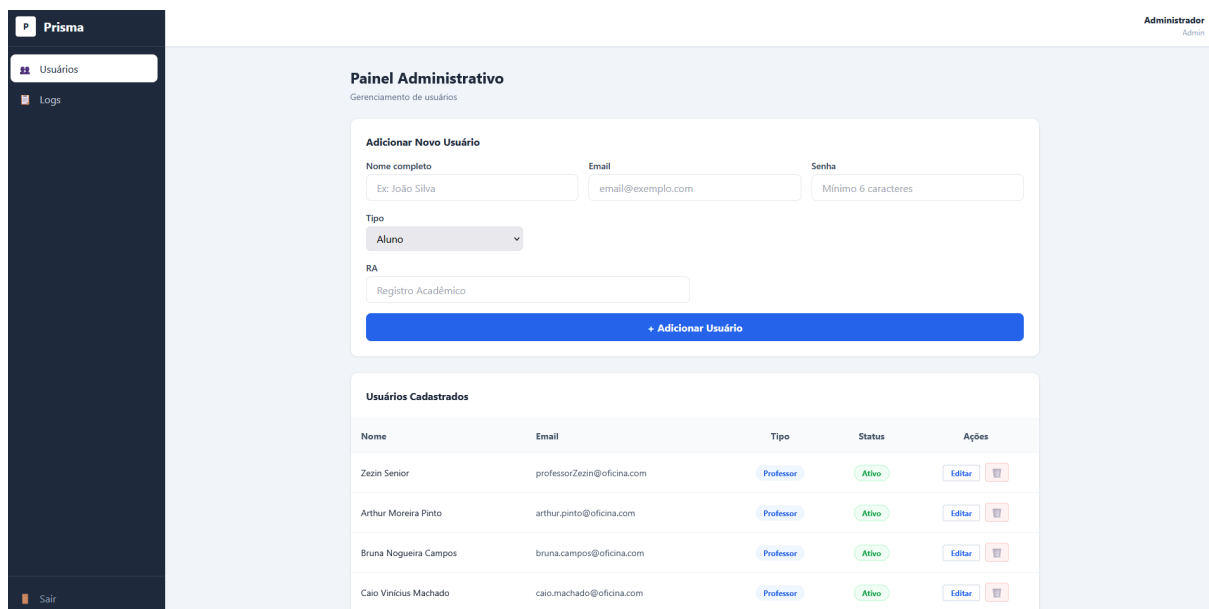


Figura 32: Painel administrativo com criação de usuários e logs do sistema.

7 Considerações Finais

7.1 Sucessos

O objetivo principal do projeto foi atingido: o sistema Prisma saiu de um conjunto de processos manuais e descentralizados para uma plataforma web integrada, com os três perfis de usuário funcionando.

No módulo do professor, o fluxo completo de cadastro de questões com assistência de IA foi implementado. O modelo classifica a disciplina, a micro-área e o nível de dificuldade na maioria dos casos, reduzindo o trabalho manual de categorização. A criação de listas a partir do banco de questões, com preservação de rascunho e marcação visual das questões já selecionadas, funcionou conforme planejado.

No módulo do aluno, o fluxo de resolução de listas e do banco livre está operacional, com registro automático das respostas e atualização dos dashboards em tempo real. O cálculo de frequência, restrito à turma principal, reflete corretamente o histórico de chamadas registradas pelo professor.

A integração entre os oito microsserviços via API Gateway funcionou de forma estável, e a arquitetura em containers Docker simplificou a execução do sistema em diferentes máquinas.

7.2 Limitações

A principal limitação operacional foi a ausência de extração automática de questões a partir de PDFs de provas. Todo o conteúdo precisa ser inserido manualmente pelo professor, o que aumenta o tempo de alimentação inicial do banco.

A acurácia da IA para classificação de dificuldade, especialmente nos níveis extremos (Muito Fácil e Muito Difícil), ainda depende de revisão humana. O modelo tende a concentrar sugestões no nível Médio para enunciados ambíguos ou curtos. Uma alternativa para melhorar a precisão seria o uso de fine-tuning supervisionado. Outra abordagem seria substituir o modelo local por uma API de linguagem de maior porte, como GPT-4o ou Claude, que tendem a apresentar melhor desempenho em tarefas de classificação sem necessidade de treinamento adicional. Ambas as alternativas foram descartadas nesta versão por restrição de recursos computacionais e de custo.

Por restrição de recursos, o sistema não foi hospedado em produção. A implantação descrita neste documento é voltada para ambiente local.

7.3 Trabalhos Futuros

As principais extensões previstas para versões futuras do sistema são:

- **Extração de questões por PDF:** implementar OCR ou parsing estruturado para importar questões diretamente de provas digitalizadas;

- **Gráficos de evolução temporal:** o banco de dados já registra o timestamp de cada resposta, o que permite construir gráficos de desempenho ao longo do tempo;
- **Notificações de frequência:** alertar automaticamente alunos próximos do limite de 75% e professores sobre alunos em risco de evasão;
- **Exportação de relatórios:** gerar PDFs com o desempenho da turma ou do aluno;
- **Suporte a imagens nas questões:** o back-end já tem a estrutura para armazenar URLs de imagens, mas a interface de upload ainda não foi integrada ao fluxo de cadastro;
- **Integração com bases públicas de vestibulares:** consumir dados públicos do ENEM e de outros vestibulares para alimentar o banco automaticamente.

8 Bibliografia

META AI. **Llama 3.2 model card**. Disponível em:
<https://ai.meta.com/blog/llama-3-2>.

OLLAMA. **Ollama documentation**. Disponível em: <https://ollama.com/docs>.

DOCKER. **Docker documentation**. Disponível em: <https://docs.docker.com>.